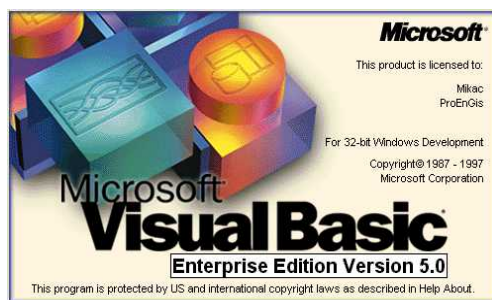


Visual Basic

(1) Osnove programiranja za Windows



R. S. Mikanović

Sadržaj

Pre početka	3
<i>O VisualBasic-u</i>	3
<i>O softveru</i>	3
<i>O priručniku</i>	4
1 Uvod u VisualBasic (1-2)	1
1.1 <i>Tipovi aplikacija koje se mogu razvijati u VisualBasic -u</i>	1
1.2 <i>Zašto programirati u VisualBasic -u</i>	1
2 Radno okruženje u VisualBasic-u (1-2)	2
2.1 <i>Elementi okruženja</i>	2
2.2 <i>Sistem padajućih menija</i>	3
2.3 <i>Paleta prečica i podešavanja</i>	3
2.4 <i>Prozor - Project</i>	4
2.5 <i>Prozor - Form Layout</i>	4
2.6 <i>Toolbox i grafički objekti</i>	4
2.7 <i>Components (Custom Controls)</i>	5
2.8 <i>Prozor Properties</i>	6
2.9 <i>Prozor kôda</i>	6
2.10 <i>Podešavanje opcija komponenata</i>	7
2.11 <i>Debug Windows</i>	7
3 Osnovni termini i pojmovi (3-4)	9
3.1 <i>Pojam objekta u VisualBasic-u</i>	9
3.2 <i>Svojstva</i>	9
3.3 <i>Metode</i>	9
3.4 <i>Događaji</i>	10
3.5 <i>Grafičke kontrole u VisualBasic-u</i>	10
3.6 <i>VisualBasic projekat i grupa</i>	11
3.7 <i>M o d u l i</i>	11
3.8 <i>Procedure</i>	15
4 Uspostavljanje sprege između korisnika i aplikacije (5-6)	16
4.1 <i>Korišćenje kontrola za unos vrednosti</i>	16
4.2 <i>Menjanje svojstava kontrola za vreme izvršavanja aplikacije</i>	16
5 Elementi programske konstrukcije (7-8)	17
5.1 <i>Principi strukturnog programiranja</i>	17
5.2 <i>Osnovne instrukcije VisualBasic -a</i>	18
5.3 <i>Osnovne funkcije VisualBasic -a</i>	19
5.4 <i>Svojstva objekata u VisualBasic-u</i>	18
5.5 <i>Operatori i njihovi prioriteti</i>	19
5.6 <i>Instrukcije grananja</i>	20
5.7 <i>Instrukcije ciklusa (petlje)</i>	22
5.8 <i>Načini i principi korišćenja instrukcije Exit</i>	23
6 Zajednički rad na izradi aplikacije (9, 10)	25
6.1 <i>Pristup i zahtevi</i>	25
6.2 <i>Redosled pri kreiranju projekta</i>	25
7 Upotreba objekata za poruke i jednostavan dijalog (11-12)	26
7.1 <i>Funkcija (i instrukcija) MsgBox</i>	26
7.2 <i>Funkcija InputBox</i>	27
7.3 <i>Visual Basic for Applications</i>	28
8 Kreiranje sistema menija (13-14)	29
8.1 <i>Prozor za kreiranje menija</i>	29
8.2 <i>Prikazivanje kôda stavke menija</i>	29
8.3 <i>Kontrolisani izlaz iz aplikacije</i>	29
9 VisualBasic Setup Wizard (13-14)	30
9.1 <i>Kreiranje izvršnog programa projekta</i>	30
9.2 <i>Provera integriteta i lokacije komponenata</i>	30
9.3 <i>Pokretanje Setup Wizard-a</i>	31
Dodatak A	33
Dodatak B	35

Pre početka

O VisualBasic -u

Microsoft VisualBasic predstavlja jedan od najpopularnijih programskih paketa za projektovanje softvera za O.S. Windows 95/98/NT, ali i njegova verzija *VisualBasic for Applications* (VBA5/VBA6), koju je Microsoft ponudio kao razvojnu i integrativnu platformu za izuzetno popularne programe iz paketa *MS Office 97/2000*, je ono čime se *VisualBasic* izdvojio od ostalih paketa. Jednostavnost, koju će ostali programerski paketi teško prevazići, je jedan od razloga njegove sve veće popularnosti kod širokog kruga korisnika PC-a pod O.S. Windows.

Prvi konkurent *MS VisualBasic*-u je *MS Visual C++*, ali je ovaj programerski paket namenjen za projektovanje aplikacija i programa najvišeg nivoa, odnosno onih koji predstavljaju delove operativnog sistema. Microsoft korporacija je, za one koji ulaze u programerske vode i žele jednostavnost i lakoću u izradi aplikacija, upravo i ponudila *VisualBasic*, a za razvoj programskih paketa nalik na *MS Office*, rezervisala *MS Visual C++*.

VisualBasic je od 1987 godine, kada je prvi put objavljena verzija VB4DOS (*VisualBasic* za DOS), prošao kroz mnoge verzije. Na našim prostorima se nije ni čulo za ovaj programski jezik do njegove verzije 3.0, koja je prvi put, nama "običnim smrtnicima" ponudila pristup grafičkim objektima i programiranju aplikacija za tadašnju verziju Windows-a (ver. 3.xx). Od tada se *VisualBasic* smatra najpopularnijim razvojnim okruženjem za Windows aplikacije, prevazilazeći svoje konkurente iz firme *Borland* (*Delphi-Pascal* i *Borland C++*) po boljoj distribuciji i plasmanu na tržištu, a *Visual C++* po pristupačnijem okruženju za one koji nemaju baš veliko iskustvo u programiranju, odnosno pristupaju ovom poslu kao hobijski ili iz želje za avanturom.

Avanturistički duh pristupa programiranju je toliko važan koliko i za video igru, pa programera gotovo nikada nećete zateći u igri, jer je programiranje za njega najzanimljivija igra. Sve više igara za Windows okruženje upravo se zasnivaju na programiranju procesa od starane igrača, a manje na destrukciji i sukobljavanju sa zamišljenim ili . Oni koji vole da svoj intelekt preokupiraju kreativnim mislima predstavljaju potencijalne programere. Sam matematički model za aplikaciju je zasigurno važan, ali će u ovom priručniku biti prikazivani samo prostiji primeri.

O softveru

Softver se danas razvija u više svetskih centara, veoma brzo i uz oštru konkurenciju, što je za krajnje korisnike svakako dobro, ali može imati i negativne posledice. Jedan od problema, koji se često sreću, je situacija da proizvođači softvera, žureći da izbacivanjem novog softvera na tržište preteknu konkurente, prave greške u samim programima, pa tako korisnicima i sebi naprave medvedu uslugu. Iako su svi, i korisnici i proizvođači, svesni ove činjenice, u budućnosti se može očekivati još veća potražnja za novim i boljim softverom, a na nama koji se bavimo njegovom proizvodnjom, je teret njegovog poboljšanja i imperativ - *program bez greške*, koji je uz današnje alate, verovatno i moguće ostvariti. I sam O.S. Windows, koji danas predstavlja najrasprostranjeniji operativni sistem, je imao sličnu istoriju, pa se definitivno pojavio posle nekoliko neuspešnih najava. Svim eventualnim programerima se preporučuje usklađivanje konačnih aplikacija sa standardom, primenjenim u poznatim i popularnim programima, a sve ono, što u ovom priručniku bude nestandardno urađeno, biće i komentarisano.

Autor je i sam sklon više kritici nego pohvali nekog, pa i lično urađenog softvera, te ga uvek daje na testiranje kolegama sa sličnim pogledom na realnu stvarnost da - *nema programa bez grešaka, ali bi ih moglo biti manje*.

O priručniku

Autor je pokušao da u ovom priručniku, prema svojim iskustvima i raspoloživom softveru, prikaže *VisualBasic* u jednom obliku, u kom će se ponuditi saveti onima koji se već bave programiranjem, ali u prvom redu zainteresovati potencijalni programeri i uticati na povećanje njihove sigurnosti u korišćenju ovog programerskog paketa.

Bez obzira što je rad u kôdu suštinski važan za programiranje u svim programskim jezicima, ovde se veći značaj pri-daje radu u korišćenju grafičkog interfejsa i automatizmima *VisualBasic*-a, dok se radom u samom kôdu zadržavamo minimalno.

Formati ispisivanja instrukcija i funkcija, dati su identično onima u *VisualBasic*-u, dok deo komande pisan malim za-košenim slovima: po principu *malaslova*, predstavlja promenljivu. Na to mesto treba uneti određenu vrednost, prema opisu argumenata. Parametri u malim zagradama () su obavezni deo komande, dok parametri u uglastim za-gradama [] nisu. Ako je prikazana sintaksa oblika { x | y }, tada treba primeniti samo jedan od argumenata: x ili y (može biti opciono dato i više argumenata).

Ovaj priručnik se koristi kao osnovna literatura za pripremu i praćenje predavanja na osnovnom kursu programiranja za *Windows* u *VisualBasic*-u, pa se autor nije trudio da detaljnije opisuje neke rutinske postupke koje je lakše i prostije dočarati na predavanjima.

Kao pripremna literatura, za upis ovog kursa i savladavanje osnovnog rada u kôdu, preporučuje se "*Priručnik za pro-gramiranje u QuickBASIC-u 7.1*" ili pohađanje kursa programiranja za *MS DOS* u *QuickBASIC*-u.

O kursu

Na osnovnom kursu programiranja za *Windows* u *VisualBasic*-u radimo po programu, čiji kratki pregled sledi (za de-taljnije upoznavanje sa planom i radom na kursu, obratite se predavaču):

- navigacija kroz *VisualBasic*, komponente interfejsa i podešavanja;
- principi i koncepcija naprednog strukturnog i objektnog programiranja;
- elementi jezika i osnove rada u kôdu *VisualBasic*-a;
- osnove programiranja na osnovu događaja,
- kreiranje nekoliko prostih aplikacija:
 - 1) za stalni prikaz datuma, vremena i sličnih informacija;
 - 2) za prevođenje iz jedne u drugu jedinicu mernog sistema i slično
 - 3) po predlogu polaznika kursa.
- svaki polaznik samostalno razvija jednu aplikaciju, od sredine do kraja rada na kursu, a njenu temu bira po svojim interesovanjima i dogovoru sa predavačem. (Na ovoj aplikaciji polaznik polaže ispit.)

I pored uloženog truda, greške u ovoj publikaciji su moguće, pa se unapred zahvaljujem svima koji mi ukažu na njih ili upute sugestije, predloge i kritike, lično ili na e-mai: **Mikac@Bits.Net**, a sve primere, koji se u toku kursa prikazuju i obrađuju, možete naći na Web strani: <http://members.nbci.com/eccosoft/source/vb/>

A u t o r

1 Uvod u VisualBasic (1-2)

Gde smo i šta možemo videti iz DEMO programa za VisualBasic

Mogućnosti VisualBasic -a

- standardan interfejs;
- lako postavljanje osobina objekata;
- rad sa OLE objektima i njihova razmena sa drugim aplikacijama;
- pristup bazama podataka različitih struktura;
- podrška Windows standard-a i jaka sprega sa O.S.-om;
- pristup svim Windows servisima,...



1.1 Tipovi aplikacija koje se mogu razvijati u VisualBasic -u

Grafički prikazi i komentari nekoliko aplikacija dati su u Dodatku A (na kraju skripta), a zbog ograničenog prostora u ovoj publikaciji, neke aplikacije će, zavisno od interesovanja polaznika, biti prikazane samo na terminima predavanja i od predavača detaljnije prokomentarisane.

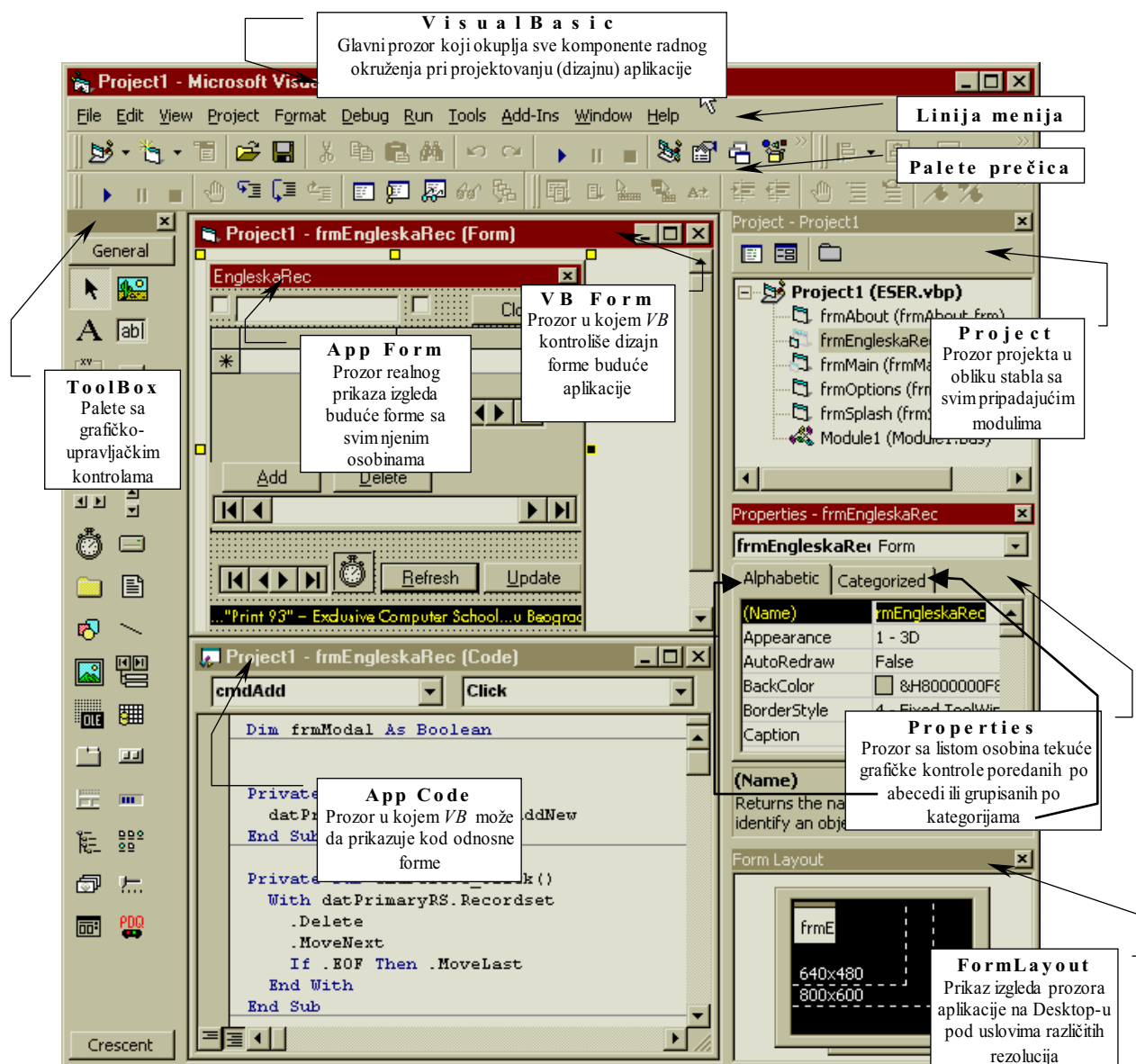
1.2 Zašto programirati u VisualBasic -u

- Jednostavnost do krajnjih granica - je jedna od odlika VisualBasic-a koja ga izdvaja od ostalih razvojnih programerskih paketa.
- Produžen Windows standard - Sve što korisnik može da vidi kao standard u Windows okruženju, može se projektovati u VisualBasic-u.
- Razvojna platforma za sve aplikacije Microsoft-a - VBA je potomak VisualBasic-a i razvijaće se u zavisnosti od razvoja ovog paketa.
- Radeći u VisualBasic-u najbolje se može sagledati i upoznati arhitektura O.S. Windows, jer su sve njegove funkcije i servisi programeru na raspolaganju, tako da se ne gubi vreme na programiranju onoga što Windows već sadrži. Kao tipičan primer ovog je korišćenje servisa: štampanja, izbor boje iz palete, izbor fonta,...
- U VisualBasic-u, i Basic-u uopšte, programeru je na raspolaganju mnogo načina da uradi isto, tako da do izražaja dolazi njegova kreativnost, od koje zavisi kvalitet i popularnost njegovog softvera.

2 Radno okruženje u VisualBasicu (I-2)

Lako je navići se na dobro okruženje

2.1 Elementi okruženja



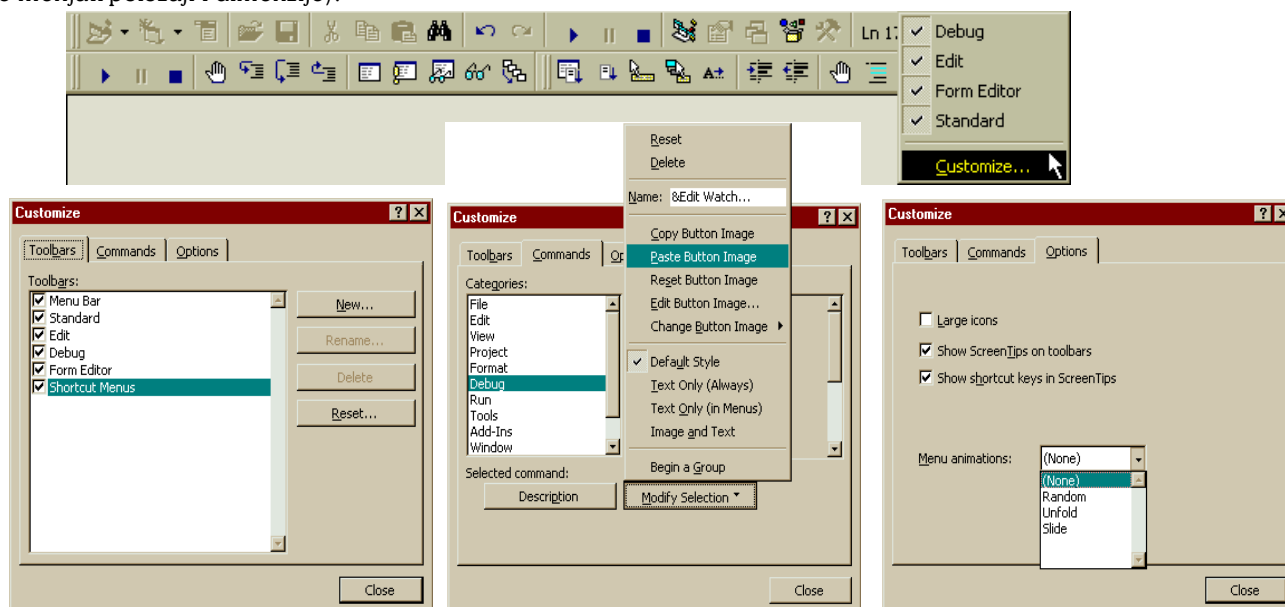
2.2 Sistem padajućih menija

Grafički prikaz kompletnog sistema padajućih menija dat je u prilogu **B**, na kraju ovog priručnika, a ovde će meniji biti opisani samo po grupisanim servisima.

File meni	sadrži pristup komandama za otvaranje, snimanje, štampanje i prevođenje (kompajliranje) <i>VisualBasic</i> projekta.
Edit meni	sadrži pristup servisima od veće pomoći za rad u kôdu nego u grafičkom dizajniranju aplikacije projekta.
View meni	sadrži mogućnosti pregleda komponenti <i>VisualBasic</i> -a, odnosno projekta.
Project meni	daje mogućnosti dodavanja i uklanjanja modula projekta, podešavanje opštih osobina projekta, kao i raspoloživih programskih biblioteka i grafičkih kontrola.
Format meni	daje mogućnosti dodatnih i naprednih alata za rad u dizajniranju grafičkog interfejsa projekta.
Debug meni	sadrži alate za rad pri testiranju i otklanjanju mogućih grešaka u aplikaciji.
Run meni	sadrži pozive komandi za start, pauzu i prekid izvršavanja aplikacije.
Tools meni	omogućava konfigurisanje programskog okruženja <i>VisualBasic</i> -a, dodavanje procedura i pregled njihovih osobina.
Add-Ins meni	obezbeđuje dodatke kojima se proširuju mogućnosti <i>VisualBasic</i> -a, odnosno povezivanje sa drugim instaliranim softverom.
Window meni	omogućava promenu tekućeg radnog prozora, kao i oblik rasporeda više prozora u radnom okruženju.
Help meni	omogućava pristup sistemu pomoći, na više kvalitetnih načina traženja pomoći o mnogim temama, terminima i postupcima.

2.3 Paleta pre čica i podešavanja

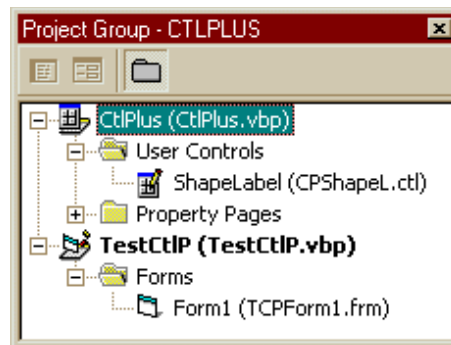
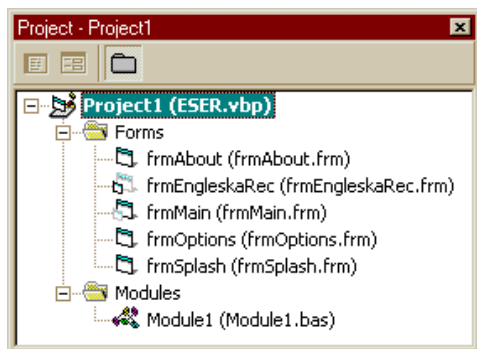
Sve komponente, osim linije padajućih menija, u radnom okruženju *VisualBasic*-a su pokretne i elastične (mogu im se menjati položaji i dimenzije).



2.4 Prozor - Project

U ovom prozoru se, u obliku stabla, prikazuju moduli koji čine tekući projekat, omogućen je pregled modula u obliku objekta (*View Object*) ili kôda (*View Code*). U korenu ovog stabla nalazi se ime projekta (*Project1*), a u zagradi ime fajla (*ESER.vbp*) u kojem je sačuvan. Dalje u stablu se razvijaju grane:

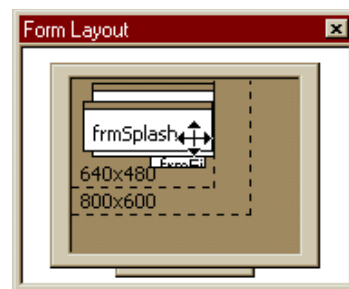
- **Forms**, u kojoj se nalaze imena i fajlovi formi,
- **Modules**, sa imenima i fajlovima standardnih modula kôda,
- **Class Modules**, sa imenima i fajlovima modula klasa.



Ako se radi na grupi projekata, stablo će biti u korenu podeljeno na projekte kao na drugoj slici desno.

2.5 Prozor - Form Layout

Sve forme, čiji su prozori za dizajn otvoreni, vide se u uprošćenom i umanjenom obliku u prozoru *Form Layout*, gde se može videti i podešavati njihov videti uzajamni odnos i položaj na *Screen*-u na nekoliko rezolucija i to od tekuće čije su granice sam prozor, pa sve do 640 x 480), tako da programer može videti kako će koja forma (pa i ceo projekat) izgledati na *Screen*-u u drugim (nižim) rezolucijama.



2.6 Toolbox i grafički objekti

Toolbox sadrži jednu ili više paleta na kojima se nalaze tipovi grafičko upravljačkih kontrola, koje je moguće uključiti u sastav forme, a preko nje i u interfejs aplikacije koja se kreira. Ako se kursom miša pristupi nekom od simbola, pojaviće se mali natpis sa imenom tipa kontrole, koju ovaj simbol predstavlja, tako da ih nije potrebno posebno pamtiti.

Toolbox je jednostavan za korišćenje i sličan interfejsu koji se primenjuje kod mnogih grafičkih programa. Kada treba postaviti neku grafičko upravljačku kontrolu na formu, najpre treba kliknuti mišem na njen simbol u *Toolbox*-u, a zatim pritisnuti taster miša na formu i "nacrtati" (razvući) odabranu kontrolu na njoj. Nanetoj kontroli se potom mogu promeniti dimenzije i/ili položaj, može se "obrisati", kopirati,... Prilikom rada na dizajniranju kontrola na formi treba koristiti mogućnosti *Drag&Drop* metode, a manje koristiti rad sa tastature, jer nije komforan.

Ovde će biti dat sažeti opis funkcionisanja nekoliko grafičkih kontrola, koje su programeru na raspolaganju po instalaciji *VisualBasic*-a. Detaljniji opis svake od ovih (ali i još nekih interesantnih) kontrola biće izložen kroz praktičan pristup rešavanju problema kreiranja sprege između korisnika i aplikacije.

Form

Predstavlja osnovnu upravljačku kontrolu na koju se postavljaju sve ostale grafičko upravljačke kontrole. U projektu aplikacije mora egzistirati bar jedna forma, da bi se na nju smestile ostale kontrole. Nova forma u projekat se dodaje preko *Add Form* stavke menija *Project*, posle čega se dobija dijalog za izbor tipa nove forme ili dodavanje fajla forme koja već postoji.

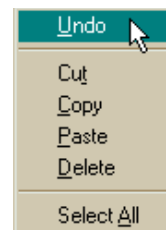
Label

Label (etiketa, oznaka,...) je do sada bila najčešće korišćena grafička kontrola i to uglavnom kao promenljiv (ili ne) komentar uz *TextBox* ili drugu grafičku kontrolu. Kako je svaka grafička kontrola, u *VisualBasic*-u 5.0, dobila svojstvo *ToolTipText*, koja se koristi za prikaz kratke poruke u situaciji kada se pokazivač miša nađe iznad odnosne kontrole, očekuje se manje korišćenje *Label* kontrole u budućnosti, čime se pored ostalog koristi i manje prostora na formi.

TextBox

TextBox je kontrola preko koje se najčešće vrši unos tekstualnih i numeričkih podataka.

U ovom okviru se, bez dodatnog programiranja, korisniku aplikacije omogućuje da koristi alate za korišćenje *Clipboard*-a. *PopUp* meni (na slici desno) se aktivira pritiskom na desni taster miša, pa korisnik treba samo da odabere jednu od ponuđenih stavki.



CommandButton

CommandButton je komandni ekranski taster i koristi se da preko njega korisnik izvrši potvrdu, otkaz i slične naloge, a aktivira se klikom mišem ili pritiskom na tastere <Enter> ili <Space>, kada je ovaj ekranski taster fokusiran.

OptionButton

OptionButton (opciono dugme) omogućava izbor jedne od ponuđenih opcija, a logička veza više ovih kontrola se automatski uspostavlja kada se one postave u jednu *Frame* ili *Form* kontrolu. Dakle, kada nekoliko grupa opcionih dugmadi, treba postaviti na istu formu, treba ih postaviti u više *Frame* kontrola prema logičkog vezi, koja među njima važi.

Frame

Frame (ram) kontrola služi za uramljivanje grupe grafičkih kontrola, a posebno je korisna kada neku grupu kontrola, treba logički vezati ili iskopirati u istu ili drugu formu.

CheckBox

CheckBox (okvir za overu) se koristi u situaciji kada korisnik treba da potvrdi ili otkáže neku predloženu stavku. Za više ovih kontrola se ne podrazumeva logička veza, već je programer uspostavlja, ako je potrebna.

ListBox

ListBox (okvir sa listom) omogućava kreiranje liste, koja se veoma često sreće kod kontrolisanog unosa podataka, odnosno kada korisniku treba ograničiti unos na neki opseg ili grupu podataka. Ovo je prva od kontrola (po datom redosledu) koja neće baš ništa raditi bez pristupa kodiranju. Da bi lista reagovala prema korisniku, na očekivani način, mora da sadrži neke stavke koje se dodaju, menjaju i uklanjaju iz kôda ili pristupom na bazu podataka. Odabrana stavka iz liste se može prepoznati po promeni pozadine teksta.

ComboBox

ComboBox (kombinovani okvir) kontrola predstavlja kombinaciju *TextBox* i *ListBox* kontrole. Kada korisnik klikne mišem na strelicu sa desne strane ovog okvira (ili u njemu pritisne kombinaciju tastera Alt+↓), aktivira padajuću listu. Kada odabere jedna od stavki iz ponuđene liste, ova stavka se automatski postavlja u gornji, fiksni deo, *ComboBox*-a. Ovo je kontrola od najvećeg značaja za dobar komfor korisnika i upravljanje aplikacijom. Pored tri tipa *ComboBox*-a sa tekstualnom listom, u današnjim najnovijim programima (*Microsoft*-a) sreću se i grafičke *ComboBox* kontrole, koje imaju visoko razvijen interfejs i proširene mogućnosti, pa se pretpostavlja da će ubrzo ova kontrola preuzeti primat koji su imale *TextBox* i *ListBox* kontrole.

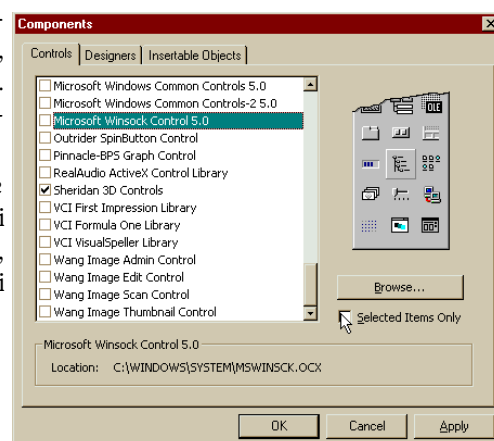
PictureBox

Ova kontrola omogućava postavljanje, na formu, gotove slike iz fajla i uključivanje u njen sastav sa, ili bez, link-a na dati fajl. Na njenom radnom prostoru omogućeno je i programirano iscertavanje korišćenjem grafičkih instrukcija.

2.7 Components (Custom Controls)

Ovaj servis se dobija desnim klikom mišem na *Toolbox* prozor i izborom stavke *Components...* ili iz menija *Project/Components...* (Ctrl+T), a omogućuje vam da proširite paletu raspoloživih grafičkih kontrola. Ovom servisu su na raspolaganju svi legalno instalirane, i servisom *References* odabrane, *.OCX biblioteke.

Dok budete vežbali rad u ovom servisu i ispitivali njegove efekte, može vam se desiti da ubacite neku od kontrola, koja vam uopšte ne treba ili vam ugrožava rad aplikacije. Da biste je uklonili iz palete *Toolbox*-a, najpre morate da je uklonite iz projekta, pa tek onda možete je ukloniti iz *Components*-a, čime je zadatak uspešno obavljen.



2.8 Prozor Properties

Kada kontroli treba detaljno postaviti svojstva koristi se *ComboBox* i lista stavki u prozoru *Properties*. Ako nijedna kontrola nije markirana prozor će biti prazan, a dok se radi u standardnom modulu kôda, ovaj prozor će sadržati samo ime modula u kome se nalazite.

2.9 Prozor kôda

Bez obzira da li se radi o modulu forme, standardnom modulu ili modulu klase, on sadrži kôd, dakle javlja se potreba da ga dodajemo, menjamo i uklanjamo.

Znanje, iskustvo i spretnost programera u pisanju kôda se razvijaju godinama rada i nemojte se razočarati kada vaši prvi programi budu "zapinjali" baš tamo gde ste vi nešto radili u kôdu. Isto tako, nije dobro da se neiskusni programer mnogo bavi radom u kôdu, jer korišćenjem nekih instrukcija i funkcija, može ugroziti integritet podataka, odnosno O.S.-a na disku.

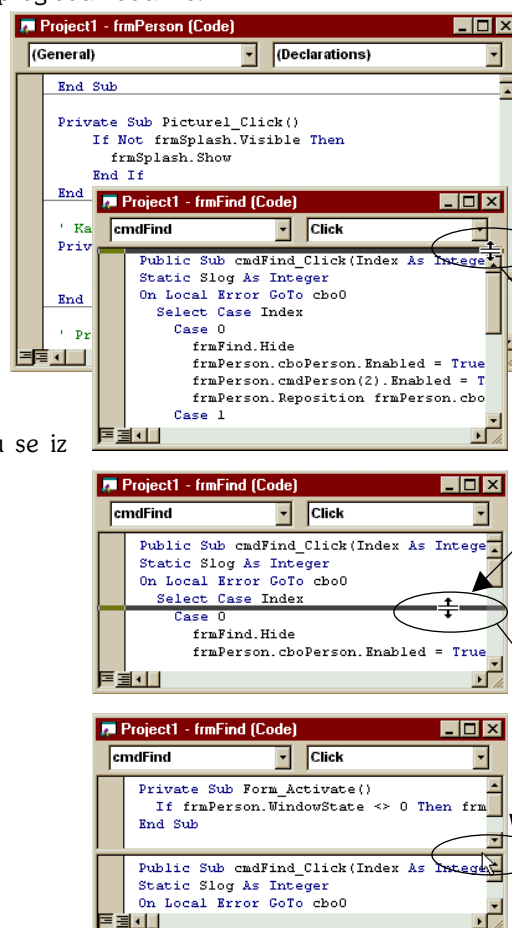
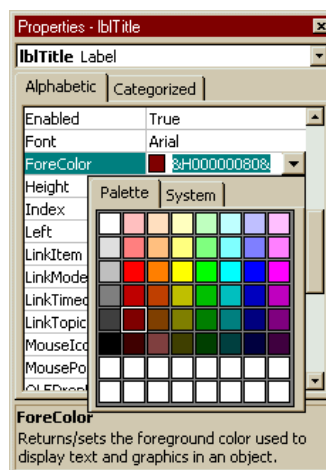
Aktiviranje ovog prozora, za neki modul, se postiže preko prozora projekta, klikom na ikonu sa natpisom *View Code*, ili *DbtClick*-om na površinu forme (ili kontrole). Na levom *ComboBox*-u, prozora kôda, biće prikazana lista objekata, a na desnom lista procedura, dok će se u slučaju standardnog modula na levom *ComboBox*-u naći samo stavka *General* koja označava početni deo modula, gde se postavljaju programske linije deklaracije podataka i eventualno biblioteka.

U donjem levom uglu ovog prozora postoje dva tastera za izbor načina pregleda kôda i to:

- levi - *Procedure View*, za pojedinačan pregled procedura (na prozoru se ne mogu naći istovremeno dve procedure) i
- desni - *Full Module View*, za pun pregled modula kôda (kada između procedura stoji tanka linija, kao separator procedure) koji se u principu mnogo više koristi.

U slučaju kada je modul kôda kompleksan (veliki broj linija) doći ćemo u situaciju da "trčkaramo" od jedne do neke druge udaljene linije u njemu, a da bi ovo izbegli treba primeniti *Split Window*, odnosno podeliti ovaj prozor kôda na dva dela, u kojima možemo istovremeno videti dva udaljena bloka programskih linija i koristiti klizače sa desne strane, da se po ovim delovima krećemo (postupak prikazan na stri slike sa desne strane).

NAPOMENA: Mnoge opcije oblika pregleda modula kôda podešavaju se iz **Options** dijaloga u karticama *Editor* i *Editor Format*.

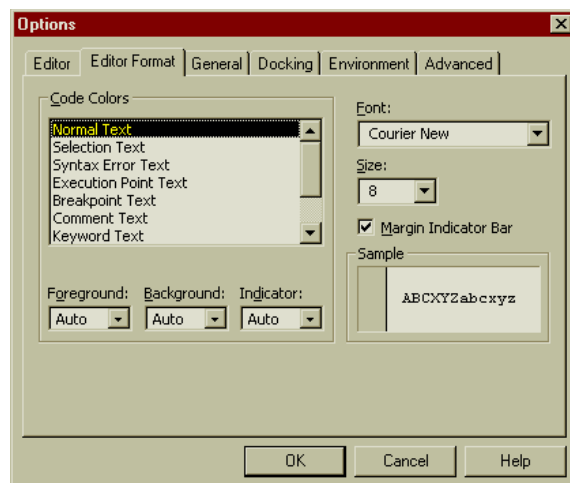


2.10 Podešavanje opcija komponenta

Da biste se na svakom računaru, dok radite u *VisualBasic-u* 5.0, osećali kao kod kuće, posvetite prve minute vašeg rada podešavanju opcija okruženja na koje ste navikli.

Zbog velikog broja kartica u dijalogu *Options* (na slici desno) ovaj deo će biti detaljno opisan na predavanjima.

Na kursu ćemo raditi prema podrazumevanim opcijama koje su postavljene instalacijom *VisualBasic-a*, uz izuzetak što ćemo stavku *Tab Width* u kartici *Editor* podesiti na 2, zbog manjeg uvlačenja pasusa prilikom rada u kodu.

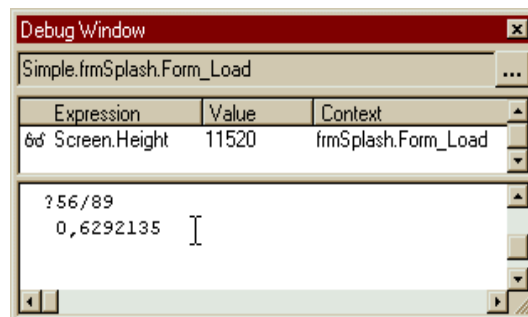


2.11 Debug Windows

Ova tri prozora (*Watch*, *Immediate* i *Locals*), koji mogu biti grafički vezani ili odvojeni, služe za specifičan pristup kompajleru u cilju otklanjanja grešaka. Kako su alati ovih prozora, u prethodnim verzijama *VisualBasic-a*, bili svedeni na jedan prozor (*Debug Window* na slici sa desne strane), sa dva dela:

- **prvi** za pregled vrednosti (gde se u našem slučaju prati vrednost izraza *Screen.Height*),
- **drugi** za direktan pristup kompajleru (gde se u našem slučaju traži ispisivanje vrednosti izraza *56/89* i daje vrednost *0,6292135*),

a kako i nekada i sada služili za otkrivanje i ispravljanje grešaka u radu aplikacije, posmatraćemo ih kao zasebne servise alata *Debugger*¹.

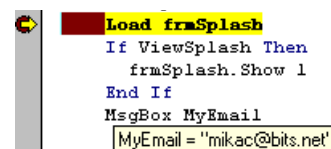


Watch

Prozor *Watch Window*, je u prethodnim verzijama predstavljao gornji deo prozora *Debug Window*, a služi za nadgledanje promene vrednosti promenljivih i opšteg toka programa. Neodvojiv je od servisa *Add Watch* i *Edit Watch*, koji se pokreću iz *Debug* menija, ali ima bržeg konkurenta *Quick Watch* (Shift+F9) koji po aktiviranju daje vrednost tekuće promenljive na kojoj se nalazi kursor ili selektovanog dela numeričkog, logičkog ili string izraza.



I od ovoga može biti brže? Pod uslovom da ste u *Options* servisu, u kartici *Editor Format* potvrdili *Auto Data Tips*, u pauzi izvršavanja aplikacije možete dobiti kratak izveštaj o vrednosti promenljive, u tekućoj proceduri, nad kojom se nalazi pokazivač miša (prikazano na slici desno).



Immediate

Sve što radimo na "podizanju" aplikacije je programski način rada, odnosno ne vidimo rezultat svoga rada dok ne pokrenemo kreiranu aplikaciju. U situaciji kada treba da sagledamo dejstvo neke instrukcije, funkcije ili vrednost promenljive (za ovo poslednje se preporučuje *Watch Window*) možemo iskoristiti mogućnost neposrednog rada.

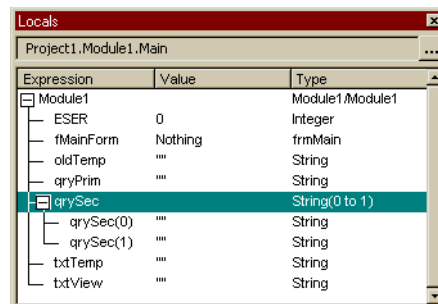
U ovom prozoru se mogu direktno zadavati instrukcije *VisualBasic-a*, ispitivati vrednosti koje daje neka funkcija i slično, što se smatra direktnim načinom rada.

¹ *Debuggerom* ćemo smatrati alat *VisualBasic-a* za pronalaženje i ispravljanje grešaka vezanih za tok odvijanja programa, iako je ovo naziv posebnog alata za pristup ispravljanju (i provociranju) grešaka u radu aplikacije (naročito u njenoj izvršnoj verziji) i nije posebno vezan ni za jedan programski jezik već svoj rad zasniva na mogućnostima operativnog sistema.

Locals

Locals prozor (na slici desno) je novina od verzije 5.0 *VisualBasic*-a i koji, na veoma komforan (*Outline*) pristup, daje pregled procedura tekućeg modula, svih izraza (promenljivih) u proceduri, njihovih vrednosti i tipa.

Pored pregleda procedura tekućeg modula, moguće je preći i na drugi modul projekta i to pritiskom na taster sa tri tačke u gornjem desnom uglu ovog prozora.



3 Osnovni termini i pojmovi (3-4)

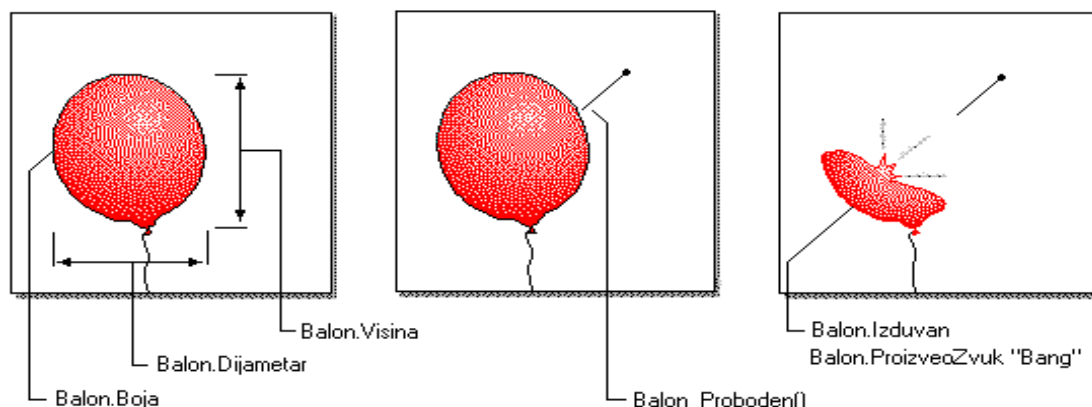
Šta je šta u VisualBasic-u

3.1 Pojam objekta u VisualBasic-u

O.S. Windows zasniva svoj rad na objektima. Ono što korisnik može da vidi su grafičko upravljачke kontrole (grafička predstava objekata), a koristeći njih uspeva da tok programa usmeri ka željenom servisu.

Svakim danom se srećemo sa aplikacijama većih mogućnosti i sa komfornijim radom, što uglavnom treba da zahvalimo ovom objektno orijentisanom operativnom sistemu, a nekada i vičnim programerima.

Svaka komponenta grafičkog interfejsa, nazvana objekat, ima tri važne grupe atributa: svojstva, metode i događaje, koji se praktično mogu predstaviti kao na sledećoj slici.



3.2 Svojstva

Svojstva (*Properties*) su oni atributi objekta koje možemo programerski ili interaktivno, u toku dizajniranja, podešavati, ali i isprogramirati njihove promene u toku rada korisnika u aplikaciji. Ponašanje objekta u aplikaciji upravo zavisi od ovih atributa. Svojstva koja se podrazumevaju, ili ne menjaju u radu aplikacije, se za dati objekat postavljaju u prozoru *Properties* kada je on aktivan.

S obzirom da svojstva uvek daju vrednost, mogu se koristiti kao funkcije, kada želite da otkrijete vrednost nekog svojstva odnosno objekta. Ako neko svojstvo objekta želimo da menjamo u toku izvršavanja programa, to moramo doprogramirati u nekom od modula (forma, standard modul ili klasa), radeći u čistom kôdu.

PRIMER 1:

```
Private Sub Form_Load()
    Me.Left = (Screen.Width - Me.Width) / 2           'Centrira tekuću formu po horizontali
    Me.Top = (Screen.Height - Me.Height) / 2          'Centrira tekuću formu po vertikali
    lblVersion.Caption = "Version " & App.Major & _
        "." & App.Minor & "." & _
        App.Revision                                  'Postavlja tekst u zaglavlje forme
    lblTitle.Caption = App.Title                      'Postavlja tekst u prvi label
    lblComments.Caption = App.Comments                'Postavlja tekst u drugi label
End Sub
```

3.3 Metode

Metod je procedura koja deluje na određenom tipu objekta (Form, TextBox, Label,...). Metod je najlakše zamisliti kao skup akcija koje taj objekat može da izvršava.

PRIMER 1:

```
Sub Main()
    frmSplash.Show           'Prikaz forme frmSplash
    frmSplash.Refresh        'Osvežavanje kontrola u formi
End Sub
```

PRIMER 2:

```
Private Sub Form_Load()
    On Local Error GoTo LoadErr
    For i = 0 To UBound(TipCD) + 1
        cboTipCD.AddItem TipCD(i)           'Dodavanje stavke u ComboBox cboTipCD
    Next
```

```

For i = 0 To UbPM
    cboMesto.AddItem RTrim(Mesto(i))      'Dodavanje stavke u ComboBox cboMesto
    cboTrans.AddItem RTrim(Mesto(i))      'Dodavanje stavke u ComboBox cboTrans
Next
For i = 31 To 1 Step -1
    lstDate(0).AddItem Format(i, "00")    'Dodavanje stavke u ListBox lstDate(0)
    lstDate(3).AddItem Format(i, "00")    'Dodavanje stavke u ListBox lstDate(3)
Next
frmMain.Top = IniVal(3)
frmMain.Left = IniVal(4)
Load frmHelp
Exit Sub

```

3.4 Događaji

Pod događajem (*Event*) se podrazumeva pretpostavljena akcija korisnika nad datim objektom. Tipičan primer za događaj je *Click* mišem na neki komandni taster. Svaki tip objekta poseduje skup akcija na koje odgovara. Šta će se dešavati posle akcije određuje programer, dodajući, u odnosnu proceduru za obradu događaja, kôd. Ime procedure za obradu događaja sadrži ime objekta, iza kog se nalazi znak podvlačenja ("_"), a za njim sledi ime događaja.

PRIMER 1:

```

Private Sub cmdMain_KeyDown(Index As Integer, KeyCode As Integer, Shift As Integer)
    ControlKey Index, KeyCode, Shift
End Sub

```

PRIMER 2:

```

Private Sub mnuHelpAutor_Click()
    AutorBox.Show 1
End Sub

```

PRIMER 3:

```

Private Sub Form_Unload(Cancel As Integer)
    IniVal(5) = frmMulti.Top
    IniVal(6) = frmMulti.Left
End Sub

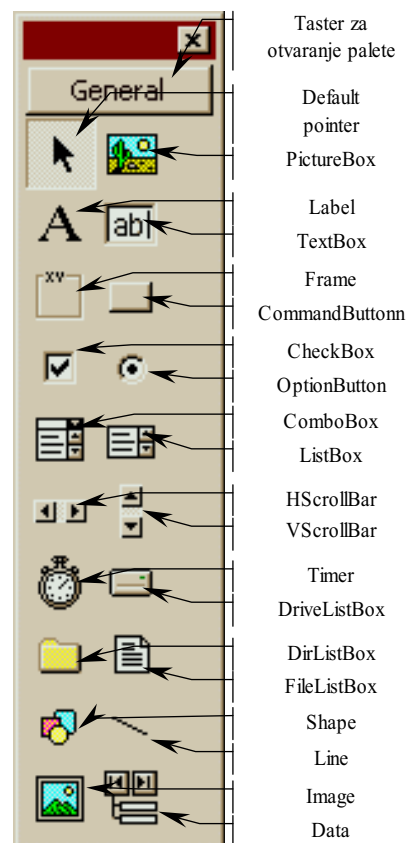
```

3.5 Grafičke kontrole u VisualBasic-u

ToolBox prozor može da sadrži više od jedne palete grafičko upravljačkih kontrolnih objekata, što zavisi od kompleksnosti aplikacija koje programer planira projektovati.

Uobičajena paleta sadrži sledeće tipove grafičkih kontrola:

Originalan naziv	Opis	Prefiks
Form	obrazac, forma	frm
Label	oznaka, nalepnica	lbl
Frame	ram, okvir	fra
CommandButton	komandni taster	cmd
CheckBox	okvir za potvrdu	chk
OptionButton	taster za opcije	opt
Image	slika	img
PictureBox	okvir slike i grafike	pct
TextBox	okvir za tekst	txt
ListBox	okvir za listanje	lst
ComboBox	kombinovani okvir	cbo
HScrollBar	horizontalni klizač	hsb
VScrollBar	vertikalni klizač	vsb
Timer	tajmer	tmr
DriveListBox	lista uređaja	drv
DirListBox	lista direktorijuma	dir
FileListBox	lista fajlova	fil
OLE object	veza na drugi softver	ole
Shape	geometrijski oblici	shp
Data	kontrola podataka	dat



Pored toga što za ime objekta treba primenjivati ista pravila koja važe i za imena promenljivih u *VisualBasic*-u, važno je držati se predloga prefiksa iz sledećih razloga:

- lakše razlikovanje imena promenljivih prilikom rada u kôdu i
- podrška zajedničkog standarda svih konvencionalnih programskih jezika.

3.6 *VisualBasic* projekat i grupa

U *VisualBasic*-u se umesto pojma aplikacija koristi sinonim projekat. Fajl projekta (*.VBP) ima zadatak da okupi sve, njemu pripadajuće, module, deklaracije korišćenih programskih biblioteka i grafičko upravljačkih kontrola, ... na jednom i konkretnom mestu.

Viši oblik okupljanja predstavlja *VisualBasic* grupu (*.VBG), gde se, opet, više projekata može grupisati, radi lakšeg kontrolisanja i zajedničkih deklaracija programskih biblioteka, pri projektovanju programskog paketa ili informacionog sistema.

Efekat korišćenja grupe projekata može se zapaziti kod instaliranja više programa na jedan računar, gde se prenose one dinamičke (DLL) i grafičko upravljačke (OCX) biblioteke, a često i baze podataka, koje su potrebne svakoj aplikaciji iz paketa, tako da ne dolazi do višekratnog prenosa jedne biblioteke ili baze podataka.

3.7 M o d u l i

Modul forme

Forma (obrazac) predstavlja primarni objekat preko koga se korisniku omogućava pristup i korišćenje grafičko-upravljačkih kontrola. Baš po ovom prozoru (*Window*) i operativni sistem (*Windows*) je dobio ime, mada nije jedini koji nudi ovakvo radno okruženje. Ovako posmatrana, forma okuplja u logičku celinu funkcionalne elemente korisničkog interfejsa u aplikaciju ili jedan njen deo.

Kako kreiramo aplikaciju za *Windows*, i ovde možemo koristiti tri osnovna oblika forme (zasnovanom na standardu *Windows* interfejsa):

- **SDI** - *Single Document Interface*, gde forma predstavlja jedan prozor, relativno prostog sadržaja (npr. kao kod aplikacije *Notepad*);
- **MDI** - *Multiple Document Interface*, je forma koja ima mogućnost okupljanja i kontrolisanja svojih potomaka, pri čemu može biti aktivna samo jedna od formi potomaka, a sistem padajućih menija koji su dodeljeni potomku preuzima MDI forma (npr. programi iz paketa *MS Office*);
- **Explore** - forma omogućava rad na dva svoja dela (panel-a), gde se akcijama na jednom utiče na dešavanja i sadržaj na onom drugom (npr. *Windows Explorer*).

Ako se u aplikaciji traži rad u više prozora naizmenično ili interakcija između prozora (npr. prenos podataka), njen interfejs treba, već od samog početka, planirati u MDI obliku, mada se i kasnije može primeniti. Jedna aplikacija ne može zasnivati svoj interfejs na više od jedne MDI forme, dok je broj njenih potomaka (SDI formi) praktično neograničen.

Kao što iole vredna aplikacija, u današnje vreme, ne može da egzistira bez izvora podataka (datoteke raznih formata), tako je za kompleksnije aplikacije bolje primeniti MDI interfejs, nego prepustiti korisniku da juri, i sam grupiše, prozore aplikacije po ekranu *Desktop*-a svog *Windows*-a. Pored toga treba mu omogućiti da, pri ponovnom startu programa, nađe prozore tamo gde ih je ostavio pri poslednjem korišćenju.

Svaka SDI forma u aplikaciji sa MDI okruženjem ne mora obavezno da bude kontrolisana kao potomak, što se naročito primenjuje na manje, i relativno kompleksne forme, čijim se pozivanjem menjaju neke opcije ili pozivaju privremeni servisi. U tom slučaju reč je o dijalozima (*Dialog Box*) koji obično nemaju sve mogućnosti kontrolisanja veličine. Ova forma se može prepoznati po tome što se njenim aktiviranjem vrši privremeno deaktiviranje glavne MDI forme u aplikaciji (npr. *Find servis* u *MS Word*-u ili *Explorer*-u).

Oblik rada u modulu forme može biti grafički pristup ili rad u kôdu forme. U kôdu forme se uglavnom radi na dodavanju programskih linija i blokova u procedure događaja, koji se odnose na samu formu ili objekte u formi.

Sama datoteka (fajl) forme je zapisana u čistom tekstualnom formatu (ekstenzije *.FRM), kome se može pristupiti iz bilo kog editor-a teksta (preporučuju se *Notepad* ili *WordPad*), ali samo u specijalnim slučajevima i sa stečenim iskustvom.

Zamislimo da treba promeniti *FontName* svojstvo svim kontrolama u formi. Najpre ćemo kreirati kopiju ovog fajla pod nekim drugim (rezervnim) imenom. Zatim otvoriti originalan fajl, koristeći *WordPad*, i izvršiti *Replace* podatka o imenu fonta, npr. "Tahoma" u "TimesNewRoman". Fajl zatim sačuvamo, pa otvorimo VBP fajl, u koji je data forma uključena, i uočimo da su sva svojstva *FontName* promenjena u "TimesNewRoman". U slučaju da dođe do problema pri učitavanju obrađenog fajla u *VisualBasic*, treba iskoristiti sačuvanu rezervnu kopiju. Greške se uglavnom dešavaju zbog brzopletih *Replace* akcija, pa više pažnje na ovom poslu neće biti na odmet.

PRIMER:

Ovde je dat sadržaj fajla forme (**frmSplash.frm**), otvorenog iz text editora. Pri pogledu na kôd iz VisualBasic-a videće se samo osenčeni deo ovog primera, dok ostalo kontroliše VisualBasic pri dizajnu forme, odnosno svojstava objekata u formi (Properties).

```
VERSION 5.00
Begin VB.Form frmSplash
    BorderStyle      = 3 'Fixed Dialog
    ClientHeight     = 5325
    ClientLeft      = 9060
    ClientTop       = 5805
    ClientWidth     = 6285
    ControlBox      = 0 'False
    LinkTopic       = "Form1"
    MaxButton       = 0 'False
    MinButton       = 0 'False
    ScaleHeight     = 5325
    ScaleWidth      = 6285
    ShowInTaskbar   = 0 'False
    Visible         = 0 'False
Begin VB.Frame fraMainFrame
    Height          = 5400
    Left            = 0
    TabIndex        = 0
    Top             = -60
    Width           = 6300
Begin VB.Data datProduct
    Appearance      = 0 'Flat
    Caption         = "Data1"
    Connect         = "Access"
    DatabaseName    = ""
    DefaultCursorType= 0 'DefaultCursor
    DefaultType     = 2 'UseODBC
    Exclusive       = 0 'False
    Height          = 315
    Left            = 330
    Options         = 0
    ReadOnly        = 0 'False
    RecordsetType   = 1 'Dynaset
    RecordSource    = "Product"
    Top             = 4380
    Visible         = 0 'False
    Width           = 1080
End
Begin VB.Label lblProduct
    AutoSize        = -1 'True
    BackStyle       = 0 'Transparent
    Caption         = "ESER"
BeginProperty Font
    Name           = "Arial"
    Size           = 32.25
    CharSet        = 0
    Weight         = 700
    Underline      = 0 'False
    Italic         = 0 'False
    Strikethrough  = 0 'False
EndProperty
    ForeColor       = &H00E0E0E0&
    Height          = 615
    Left            = 3840
    TabIndex        = 16
    Tag             = "Product"
    Top             = 1200
    Width           = 1770
End
...
End
End
```

```
Attribute VB_Name = "frmSplash"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = True
Attribute VB_Exposed = False

Dim Korak As Integer
Private Sub cmdClose_Click()
    Timer1.Enabled = False
    'Load fMainForm
    frmMain.Show
    Me.Hide
End Sub

Private Sub Form_Load()
    Me.Left = (Screen.Width - Me.Width) / 2
    Me.Top = (Screen.Height - Me.Height) / 2
    Korak = 50
    datProduct.DatabaseName = App.Path & " \ESER.mde"
    lblVersion.Caption = "Version " & App.Major &
        "." & App.Minor
    lblProductName.Caption = App.Title
End Sub

Private Sub pctLogo_Resize()
    Select Case pctLogo.Width
        Case Is >= 6255, Is < Abs(Korak)
            Korak = -Korak
    End Select
End Sub

Private Sub Timer1_Timer()
    pctLogo.Width = pctLogo.Width + Korak
    Me.Refresh
End Sub
```


Standardan modul kôda

Standardan modul kôda sadrži uglavnom procedure i deklaracije promenljivih, a koje treba da budu na raspolaganju svim procedurama projekta. Ovaj modul se, uz minimalne izmene u deklarisanju, može uvesti i iz *QuickBASIC*-a za DOS, uz uslov da je programer poštovao principe strukturnog i proceduralnog programiranja. Jedan projekat ne mora da sadrži nijedan standardni modul kôda, a kompleksniji projekti mogu imati i više od jednog.

Sve procedure, koje se koriste iz više formi, mogu se postaviti u ovaj modul, čime se štedi na veličini samog projekta, smanjuje kompleksnost kôda forme, ubrzava rad aplikacije i sl.

Kada se javi potreba da se izvesna grupa procedura izdvoji, kao logička ili funkcionalna celina, kreira se novi standardni modul i sve ove procedure prebace u njega. Jedan standardni modul kôda može se koristiti istovremeno u više projekata, čime programer može da uštedi vreme i prostor na disku.

PRIMER: Standardni modul kôda *Timen.bas*

```
' Timen.BAS - Mikac - (C) 1996, DuRadAn Software. ALL RIGHTS RESERVED.
Option Explicit
Global Const UbSeg = 32, ExINI = ".INI", ExHLP = ".HLP", ExDDT = ".DDT"
Global Const ExINF = ".INF", ExTXT = ".TXT", MyProgramINI = "Timen30.INI"

Public Type SlogTyp
    Prezime As String * 20
    Ime As String * 15
    Zvanje As String * 15
    TelStan As String * 15
    Firma As String * 40
    Funkcija As String * 20
    TelFirma As String * 25
    Fax As String * 20
    Adresa As String * 25
    Mesto As String * 25
End Type

Public MyPath '= App.Path + "\"
Public MyDesk '= MyPath + "Program \Desk.ini"
Public MyPrnSet
Public MyRecord As SlogTyp
Public TRecord As SlogTyp
Public TempRecord As SlogTyp
Public PrazRecord As SlogTyp
Public OldRecord As SlogTyp
Public CopRecord As SlogTyp
Public SetGro(5) As Integer
Public RecordNiz(1 To 10) As String

'Za print setup
Public PsA(20) As Variant, PsL(20) As Variant, PsN(20) As Variant
Public PsALN As Integer
Dim Tool
Dim i As Integer, s As String, Ok1 As Integer, Ok2 As Integer, FlN As Integer

Sub BaseInput(X As Long)
    On Error Resume Next
    Open MyDataBase(1) + ExDDT For Random Access Read Write Shared As #1 Len = Len(MyRecord)
    Get #1, X, MyRecord
    Close #1
End Sub

Function BezBlank(s As String) As String
    BezBlank = Trim(s)
End Function

Public Sub CtrlInterval()
    Select Case Korak
        Case 1
            If TekInterval > UbSeg Then
                TekInterval = 1
            End If
        Case -1
            If TekInterval < 1 Then
                TekInterval = UbSeg
            End If
    End Select
End Sub

Sub CtrlItem(Poc As Integer, Uk As Integer, It As Integer, Za As Integer)
    It = It + Za
    If It > Uk Then It = Poc
    If It < Poc Then It = Uk
End Sub
```

Modul klase

Objektno orjentisano programiranje podrazumeva rad sa klasama objekata. Ovo je tema koja se obrađuje na višem kursu programiranja za *Windows* u *VisualBasic*-u, a ovde će biti prikazan samo jedan primer modula klase.

PRIMER: Modul klase *CApplication.cls*

Option Explicit

```
' Omotac za poruku o autorskim pravima
Public Property Get Copyright() As String
    Copyright = App.LegalCopyright
End Property
' Datoteka za pomoc
Public Property Get HelpFile() As String
    HelpFile = Path & App.HelpFile
End Property
```

```
' Putanja
' Obezbedjuje da postoji "\" na kraju
Public Property Get Path() As String
    If Right(App.Path, 1) = "\" Then
        Path = App.Path
    Else
        Path = App.Path & "\"
    End If
End Property

' Omotac za naslov
Public Property Get Title() As String
    Title = App.Title
End Property
```

3.8 Procedure

Opšti sinonim procedure je logička celina nekoliko programskih linija koje obavljaju neki često korišćeni posao ili izračunavaju vrednost funkcije, pa se još nazivaju i potprogrami. Ako se neki potprogram nalazi i poziva u izolovanom logičkom sklopu onda je on unutrašnji potprogram i nije na raspolaganju drugim celinama programa, a ako je kreiran u obliku procedure, može (ali nije obavezno) da bude na raspolaganju svim ostalim delovima programa) moduli, procedure, itd.

Pod procedurama se u *VisualBasic*-u sreću dve vrste spoljašnjih potprograma i to Sub i Function. Svi principi programiranja pod DOS-om važe i ovde, s tom razlikom što se uvek očekuje eksplicitna deklaracija tipa kod vrednosti Function procedure.

Ako poštujete hijerarhiju strukturnog programiranja, sve ono što je zajedničko za više procedura smestićete u jednu zasebnu proceduru, koja će njima biti na raspolaganju, a ono što bi se na više mesta ponavljalo u samo jednoj proceduri, smestićete u unutrašnji potprogram (i pozivati sa GoSub ili On..GoSub).

Sub procedure

Ovo su procedure kojima se prosleđuje (ali nije obavezno) jedan ili više argumenata, i koje prema njihovim vrednostima obavljaju izvesne zadatke.

Function procedure

Pored toga što funkcioniše na isti način kao i Sub procedura, Function procedura, u svom imenu, ima mogućnost da vrati vrednost (kao i svaka standardna funkcija *VisualBasic*-a). Prema tome, *Function* procedura se smatra boljom za korišćenje kada treba odrediti vrednost jedne promenljive prema vrednostima jednog ili više argumenata. U programima, gde treba detaljno servisirati greške ova procedura se koristi kao indikator greške, a ako je njena vrednost posle obavljenog zadatka različita od 0 to znači da je u izvršavanju došlo do greške čiji kod odgovara vrednosti te funkcije (jasno je da greška sa kôdom 0 ne postoji).

Even-Driven procedure

Svaka grafička kontrola, ugrađena u formu "nosi" sa sobom i svoje procedure događaja, pa tako korisnik u korišćenju aplikacije nema linearni tok diktiranog odvijanja aplikacije, već može koristiti mogućnosti događaja za sve grafičke kontrole na datoj formi aplikacije. U kojoj meri će programer uspeti da uputi korisnika aplikacije da intuitivno (ali ipak pod uticajem izgleda neke kontrole) vrši samo one akcije koje su servisirane događajem, a ne one druge, stvar je osećaja kod programera. Programiranjem svih događaja, odnosno presretanjem akcija korisnika nad datom grafičkom kontrolom, može se kod korisnika stvoriti suprotan efekat, tj. da od šume informacija ne vidi onu koja mu je najbitnija.

Obično se za neku grafičku kontrolu isprogramira 20-30% očekivanih akcija korisnika, dok se sve ostale usmere da koriste već postojeće procedure događaja.

Sledeće teme biće obrađene samo na predavanjima:

Koncepcije programiranja i projektovanja:

- Modularno programiranje
- Proceduralno programiranje
- Objektno orjentisano programiranje
- Upravljanje događajima

Kreiranje novog projekta:

- Okupljanje modula projekta
- Deljenje zajedničkih modula
- Upravljanje delovima projekta

4 Uspostavljanje sprege između korisnika i aplikacije (5-6)

Pretpostavimo da korisnik prvi put vidi računar

4.1 Korišćenje kontrola za unos vrednosti

TextBox

```
Private Sub txtTehnik_Change(Index As Integer)
    If Len(BezBlank(txtTehnik(0).Text + txtTehnik(1).Text + txtTehnik(2).Text)) Then
        cmdTehnik(5).Enabled = True
    Else
        cmdTehnik(5).Enabled = False
    End If
End Sub

Private Sub txtTehnik_KeyDown(Index As Integer, KeyCode As Integer, Shift As Integer)
    If Not IsNew And KeyCode = 13 Then Korekcije
End Sub
```

OptionButton

```
Private Sub optKriterP_Click(Index As Integer)
    Select Case Index
        Case 0
            Kriter(0) = 0
        Case 1
            Kriter(0) = 1
    End Select
End Sub

Private Sub optKriterS_Click(Index As Integer)
    Select Case Index
        Case 0
            Kriter(1) = 0
        Case 1
            Kriter(1) = 1
    End Select
End Sub
```

CheckBox

```
Private Sub Check1_Click()
    AutoRefresh = Check1.Value
End Sub

Private Sub Check1_MouseMove(Button As Integer, Shift As Integer, X As Single, Y As Single)
    STB ("Potvrdite opciju ako želite automatsko prikazivanje preseka posle svake promene
    u podacima ili dimenziji slike.")
End Sub
```

4.2 Menjanje svojstava kontrola za vreme izvršavanja aplikacije

Ovde ćemo pokušati da zajedno napravimo nekoliko jednostavnih aplikacija:

- sat sa konstantnim prikazom datuma i vremena,
- aplikaciju za prevođenje iz jedne u drugu mernu jedinicu iste fizičke veličine,
- aplikaciju za rešavanje nekih tipskih zadataka iz matematike i fizike,
- potom će svako od vas da odluči koju će problematiku da obradi u svojoj zasebnoj aplikaciji, a onda to sve zabeleži u nastavku ove strane.

Podsetite se:

$$1^\circ = (\pi/180) \text{ rad}$$

$$1' = (\pi/10800) \text{ rad}$$

$$1'' = (\pi/648000) \text{ rad}$$

$$\pi = 22/7$$

$$g = 9,806$$

$$1 \text{ kcal} = 1/860 \text{ kWh} = 4186,84 \text{ J}$$

$$1 \text{ inch} = 25,4 \text{ mm}$$

$$1 \text{ stopa} = 0,3048 \text{ m}$$

$$1 \text{ milja} = 1,609 \text{ km}$$

Površine geometrijskih slika i tela:

Zapremina geometrijskih tela:

5 Elementi programske konstrukcije (7-8)

Ono čime vi preuzimate kontrolu nad svim delovima projekta

5.1 Principi strukturnog programiranja

Strukturno programiranje je specifičan način kreiranja programa (u kôdu) kod koga se za kreiranje logičke programske strukture primenjuju standardizovani elementi. Strukturno programiranje je konceptijski postavljena tehnika projektovanja programa u skladu sa skupom utvrđenih pravila, kojima se umanjuje mogućnost pojave problema kod testiranja, povećava produktivnost i čitljivost napisanog programskog kôda. Primena tehnike strukturnog programiranja pretpostavlja eliminisanje **GoTo** naredbe i uvođenje utvrđenih logičkih algoritamskih struktura sa adekvatnim strukturnim instrukcijama grananja i kontrolnim instrukcijama. Takođe, tehnika strukturnog programiranja uključuje koncepciju oblikovanja programa tehnikom modularnog programiranja i razvoja svrha, kao i utvrđeni broj manje važnih ograničenja i pravila u odnosu na programiranje.

Kritična faza u razvoju softvera je testiranje. Pokazalo se da su troškovi testiranja i modifikacije, nekonvencionalno urađenog programa, 100 puta veći od troškova izrade samog programa. Činjenica je da mnoge greške koje se ne otkriju u fazi testiranja, kasnije u eksploataciji dovode do ogromnih finansijskih gubitaka ne samo proizvođača, nego i korisnika. Kraće vreme testiranja, kao i rezultati u boljoj jasnoći i preglednosti, dovode do povećanja produktivnosti programera. Skraćenje vremena izrade programa je jedan od preduslova konkurentnosti na tržištu softvera.

Pored ovoga, na kvalitet programa utiče i brzina njegovog izvođenja, što opet zavisi od kompajlera u kome je softver rađen, ali i usklađenosti sa operativnim sistemom. Po ovom pitanju programi danas nemaju većih problema, jer se svakim danom pojavljuju novi računari PC konfiguracije sa još boljim performansama.

Osnovni princip modularnog programiranja je u tome da program treba razložiti na sitnije celine, sve dok se ne dođe do sekvenci koje su pregledne i koje se mogu lako kodirati. Obično, ova sekvenca, ne bi trebalo da pređe visinu jedne stranice kôda prikazanog na ekranu. Na taj način, ako dokažemo ispravnost modula na k -tom nivou, lako ćemo dokazati njegovu ispravnost i na $k+1$ nivou. Sledeći hijerarhijsku organizaciju modula, za veoma kratko vreme, možemo odrediti koji od njih neispravno funkcioniše.

Najvažnije prednosti modularnog programiranja su sledeće:

- moduli se mogu testirati nezavisno. Proveren i iztestiran modul se prilaže projektu ili programskoj biblioteci.
- jedan modul istovremeno se može koristiti u više projekata,
- održavanje projekata je olakšano s obzirom da su procedure podeljene po modulima, a moduli po nameni.

Kod rada sa modulima treba se držati opštih pravila:

- svaki modul treba da sadrži najmanje jednu nezavisnu proceduru,
- veze između modula moraju biti jednostavne i lako prepoznatljive,
- svi podaci koji se koriste samo u datom modulu moraju biti tako i deklarirani, odnosno ne smeju biti na raspolaganju drugim modulima,
- modul treba da utiče samo na izmenu definisanih podataka, a da nema uticaja na globalni tok programa,
- logička struktura modula mora biti pregledna.

Teorija na kojoj se zasniva tehnika strukturnog programiranja, zasniva se na tvrđenju da se svaka logička programska celina može proizvesti od tri osnovne strukture:

Sekvenca - elementarna algoritamska struktura (nekoliko uzastopnih instrukcija)

Selekcija - razgranata algoritamska struktura (uslovno grananje), kojom se obezbeđuje izbor između (najmanje) dva alternativna puta, jedan kada je uslov zadovoljen, a drugi kada nije. Ovde se mogu svrstati instrukcije *If..Then..Else*, *On..GoSub..*, *Select Case..*, itd.

Iteracija - ciklična algoritamska struktura (višestruko ponavljanje niza operacija), pri čemu je broj puta ponavljanja ograničen datim uslovom. Ovde se mogu primeniti petlje: *Do..Loop*, *Do..While*, *While..Wend*, *For..Next*, itd.

Procedura koja je kreirana tehnikom strukturnog programiranja, mora da zadovolji sledeće kriterijume:

- svaki logički tok može imati samo jednu ulaznu i jednu izlaznu tačku,
- logički tok procedure se vrši od ulazne do izlazne tačke uz korišćenje i povezivanje različitih elementarnih algoritamskih koraka, zavisno od akcija korisnika.

Preglednost programa u mnogome zavisi i od stila pisanja kôda. Preporučljivo je da se pojedine funkcionalne celine zapisuju u "bloku" (jedno ispod drugog). Kôd postaje čitljiviji ako su programske linije unutrašnjeg ciklusa uvučene u

odnosu na linije spoljašnjeg ciklusa. Pored toga, veoma je značajno prokomentarisati modul, proceduru, ali i neki programski blok.

VisualBasic ima nekoliko osnovnih programskih konstrukcija (struktura i simboličkih elemenata) koji grade programske blokove kôda, a obuhvataju: instrukcije, funkcije, procedure, metode, svojstva, događaje i operatore.

5.2 Osnovne instrukcije *VisualBasic* -a

Beep	Exit	Open	Option Private
Call	FileCopy	Option Base	Print #
ChDir	For...Next	Option Compare	Private
ChDrive	Function	Option Explicit	Public
Close	Get	Do...Loop	Put
Const	GoSub...Return	End	Randomize
Date	GoTo	Erase	ReDim
Declare	If...Then...Else	Error	Rem
DefBool	Input #	Width #	Resume
DefByte	Kill	With	Rmdir
DefCur	Let	Write #	RSet
DefDate	Line Input #	DefVar	Select Case
DefDbl	Load	Dim	Set
DefInt	LSet	On GoSub	Static
DefLng	Mid	On Goto	Stop
DefObj	Mkdir	Unload	Sub
DefSng	Name	While...Wend	Time
DefStr	On Error	Write	Type
Abs	FileAttr	Day	QBColor

5.3 Svojstva objekata u *VisualBasic* -u

Align	Color	FontCount	Weight
Alignment	Cols	FontItalic	Width
AllowAddNew	ColumnCount	FontName	WindowState
AllowDelete	ColumnHeaders	Fonts	MaxLength
AllowRowSizing	ColumnName	FontSize	MDIChild
AllowSizing	ColWidth	FontStrikethru	MinButton
AllowUpdate	Connect	FontTransparent	MultiLine
AutoActivate	ControlBox	FontUnderline	MultiSelect
AutoRedraw	CurrentY	ForeColor	Name
AutoShowChildren	Database	Height	Orientation
AutoSize	DatabaseName	Icon	PaperSize
BackColor	DataField	Image	PasswordChar
BackStyle	DataSource	Index	Picture
BOFAction	Default	IniPath	RecordSource
Bold	DeviceName	InitDir	Row
BorderColor	DrawStyle	Interval	RowIsVisible
BorderStyle	DrawWidth	Italic	Rows
BorderWidth	Drive	ItemData	RowSource
BoundColumn	DriverName	Left	ScrollBars
Cancel	Enabled	List	ShowInTaskbar
CancelError	Exclusive	ListCount	Sort
Caption	EXENAME	ListIndex	Sorted
ClipControls	FillColor	Visible	Source
Col	FillStyle	VisibleCols	TabStop
ColAlignment	Font	VisibleCount	Text
ColIsVisible	FontBold	VisibleRows	Top

5.4 Osnovne funkcije VisualBasic -a

Asc	FileDateTime	Dir	Right
AscB	FileLen	EOF	Rnd
Atn	Fix	Error	RTrim
CBool	Format	Exp	Second
CByte	FreeFile	Minute	Sgn
CCur	Hex	Month	Shell
CDate	Hour	MsgBox	Sin
CDBl	IIf	Now	Space
Chr	Input	Oct	Spc
CInt	InStr	Trim	Sqr
CLng	Int	UBound	Str
Cos	LBound	UCase	String
CSng	LCase	Val	Tab
CStr	Left	Year	Tan
CurDir	Len	DateValue	Time
Date	Log	Mid	Timer
DateSerial	LTrim	TimeValue	TimeSerial

5.5 Operatori i njihovi prioriteti

Operator obavlja aritmetičke operacije, te logička i relacionala testiranja. Kao i kod drugih programskih jezika, ovi operatori imaju svoje prioritete izvršavanja, pa će tim redom biti i opisani:

Tip	Simbol	Opis	Prioritet
Aritmetički	^	stepenovanje	1
	-	negativni predznak	2
	*	množenje	3
	/	deljenje	3
	\	celobrojno deljenje	4
	Mod	aritmetički moduo	5
	+	sabiranje	6
	-	oduzimanje	6
Relacioni	=	jednako	8
	>	veće od	9
	<	manje od	9
	<=	manje ili jednako	9
	>=	veće ili jednako	9
	<>	različito	9
	Is	ispituje tip objekta	9
Logički	Not	negacija (NE)	9
	And	konjunkcija (I)	10
	Or	disjunkcija (ILI)	11
	Xor	isključivo ILI	12
	Eqv	ekvivalencija	13
	Imp	implikacija	14
String operatori	&	nadovezivanje stringova	7
	+	sabiranje stringova	7
	Like	poređenje stringova	9

5.6 Instrukcije grananja

Prosto grananje - If..Then..Else

Instrukcija **If..Then..Else** omogućava najprostiji oblik grananja u programskoj strukturi.

SINTAKSA:

```
If izraz Then instr1 [Else instr2]
                        ili
If uslov1 Then
    [blok1]
[ElseIf uslov2 Then
    [blok2]]
...
[Else
    [blok3]]
End If
```

PRIMER:

```
If RTrim(s) = "[" + Section + "]" Then
    Do
        If InStr(s, Key) Then
            InLine = RTrim(Mid(s, InStr(s, "=") + 1, Len(s)))
            ReadIni = Len(InLine)
            Exit Do
        End If
        Input #Filnum, s
    Loop Until InStr(s, "[") Or EOF(Filnum)
End If
```

Prelazak na unutrašnji potprogram instrukcijom GoSub

Instrukcija **GoSub** usmerava program na izvršavanje unutrašnjeg potprograma.

SINTAKSA: GoSub lokacija

lokacija - broj ili oznaka programske linije za grananje.

NAPOMENA: **Return** instrukcija se koristi za povratak iz unutrašnjeg potprograma. Ako se neka rutina (skup instrukcije) ponavlja više puta u proceduri, ima smisla kreirati unutrašnji potprogram, a ako se neka rutina ponavlja u više procedura ili modula, treba kreirati novu proceduru (Sub ili Function) za izvršavanje ove rutine.

PRIMER:

```
Private Sub Form_Load()
    Dim MsgText As String
    MsgTitle = "Centrall ver. 1.02."
    On Local Error Resume Next
    Load frmInfo
    If PasswordL < (Val(YourPassword) - Date2L(USADate) + DatumYU2L(FirstDate)) Then
        GoSub NoLegal
    ElseIf DatumYU2L(FirstDate) > Date2L(USADate) Then
        GoSub NoLegal
    End If
    Label1.Left = 2500
    Label2.Left = -2500
    Label3.Left = 4000
    Label4.Left = -2500
Exit Sub
NoLegal:
    MsgBox "Program koristite NELEGALNO ili prvi put!" & _
        vbCrLf & "Pozovite autora da biste legalizovali kopiju!", vbCritical, MsgTitle
    frmInfo.Show 1
End
End Sub
```

Višestruko grananje instrukcijom On..GoSub..

On..GoSub instrukcija grana na jednu od navedenih lokacija na osnovu rezultata izraza.

SINTAKSA: **ON** *numizr* **GOSUB** *lokacija* [, *lokacija*]...

gde je:

- numizr* bilo koji numerički izraz u opsegu od 0 do 255.
- Ako je potrebno izraz se zaokružuje na celobrojnu vrednost.
- lokacija* lista brojeva linija ili labela na koje se grana.
- Ako je vrednost numeričkog izraza 1 grananje ide na prvu lokaciju. Ako je vrednost numeričkog izraza 2 grananje ide na drugu lokaciju itd.
- Ako vrednost nema odgovarajuću labelu izvršavanje se nastavlja na sledećoj instrukciji.

PRIMER:

```
Sub Primer(i As Integer)
  For i = 1 To 3
    On i GoSub Jedan, Dva, Tri
  Next
Exit Sub
Jedan:
  MsgBox "U Jedan"
  Return
Dva:
  MsgBox "U Dva"
  Return
Tri:
  MsgBox "U Tri"
  Return
End Sub
```

Napredno grananje - Select Case

Instrukcija **Select Case** vrednuje i izvršava odgovarajući blok instrukcija.

SINTAKSA:

```
Select Case testizraz
  Case uslov1
    [blok1]
  [Case uslov2]
    [blok2]
  ...
  [Case Else]
    [default.blok]
```

End Select

- instrukcije* lista instrukcija koje se izvršavaju u slučaju uparivanja;
- default.blok* lista instrukcija koje se izvršavaju kad nije pronađen nijedan izraz za uparivanje.
- Ove instrukcije su pridružene **Case Else** klauzuli.
- testizraz* string ili numerički izraz za vrednovanje i upoređivanje sa mogućim slučajevima.
- uslov1,2* izraz za uparivanje test izraza.

NAPOMENA: **Select Case** vrednuje izraz i pretražuje listu mogućih slučajeva za uparivanje. Ako je uslov ispunjen izvršavaju se instrukcije za taj slučaj.

Izraz za uparivanje može biti zapisan u tri oblika:

Prvi oblik ***izraz1* [, *izraz2*]**... Ako jedan od izlistanih izraza ima vrednost kao i *testizraz*, izvršava se, njemu pripadajući, blok instrukcija.

Drugi oblik Izraz uparivanja može takođe imati i oblik opsega: ***izraz1 To izraz2***, što omogućava rangiranje mogućih vrednosti za uparivanje.

Treći oblik Izraz uparivanja može koristiti i relacione operatore i to na način: ***Is relacioni.operator izraz***, gde je *relacioni.operator* <, >, <=, >=, =, <>. *VisualBasic* upisuje ključnu reč **Is** ako je izostavite. Treba skrenuti pažnju da u ovom načinu, kao ni u prethodna dva, *testizraz* ne sme da se pojavljuje u izrazima *uslov1, uslov2*...

PRIMER:

```
Private Sub cboUpo_Click(Index As Integer)
  Select Case Index
```



```

Case 0
  If cboUp0(0).ListIndex = cboUp0(1).ListIndex Then Exit Sub
Case 1
  If cboUp0(1).ListIndex = cboUp0(0).ListIndex Then Exit Sub
Case 2
  If Not Start Then LoadList 1
  Exit Sub
End Select
If Not Start Then LoadList 2
End Sub

```

5.7 Instrukcije ciklusa (petlje)

Indeksirana iteracija - For..Next

Instrukcija **For..Next** Ponavlja dati skup instrukcija dozvoljeni broj puta.

SINTAKSA:

```

For kontrvar = pocvredn To krajvredn [Step korak]
  [instrukcije]
Next [kontrvar[, kontrvar]...]

```

kontrvar promenljiva koju **For** povećava svakim prolazom;
pocvredn početna vrednost kontrolne promenljive;
krajvredn vrednost koju kontrolna promenljiva mora imati da bi se petlja zatvorila;
korak vrednost koja se dodaje kontrolnoj promenljivoj u svakom prolazu (inkrement). Može biti pozitivna ili negativna. Ako se ne navede ništa podrazumeva se da je 1.

NAPOMENA: **Next** instrukcija usmerava na povećanje kontrolne promenljive i testira da li je ona veća od krajnje zadate vrednosti. Ako nije, izvršavanje se nastavlja na prvoj instrukciji unutar petlje, a ako jeste, izvršavanje se nastavlja na prvoj instrukciji iza **Next**.

PRIMER:

```

For n = 0 To (lstTop.ListCount - 1)
  If lstTop.Selected(n) = True Then
    lstTop.Selected(n) = False
    For M = 0 To (lstList123.ListCount - 1)
      If lstList123.List(M) = lstTop.List(n) Then
        M = -1
        Exit For
      End If
    Next
    If M <> -1 Then lstList123.AddItem lstTop.List(n)
  End If
Next

```

Prosta iteracija - While..Wend

While..Wend je petlja koja ponavlja skup instrukcija dok je dati uslov zadovoljen.

```

Private Sub Form_Load()
  frmHelp.Top = IniVal(9)
  frmHelp.Left = IniVal(10)
  Filnum = FreeFile
  Open ValidPath + "HELPK.hlt" For Input As Filnum
  While Not EOF(Filnum)
    Line Input #Filnum, s
    lstHelpK.AddItem s
  Wend
  Close Filnum
End Sub

```

Napredna iteracija - Do..Loop

Ponavlja skup instrukcija sve dok uslov ne dobije istinitu vrednost.

SINTAKSA:

Do Until <i>Bulov.izraz</i>	ili	Do
[instrukcije]		[instrukcije]
Loop		Loop Until <i>Bulov.izraz</i>

Ponavlja skup instrukcija sve dok je zadovoljen dati uslov.

SINTAKSA:

Do While <i>BulovIzraz</i>	ili	Do
[instrukcije]		[instrukcije]

Loop**Loop While** *BulovIzraz*

Gde je *Bulov.izraz* bilo koji izraz koji ima vrednost istina (tačan, True) ili laž (netačan, False).

Objedinjeno, sintaksa bi izgledala ovako:

```
Do [{While | Until} Bulov.izraz]
  [instrukcije]
[Exit Do]
[instrukcije]
Loop
```

ili ovako

```
Do
  [instrukcije]
[Exit Do]
[instrukcije]
Loop [{While | Until} Bulov.izraz]
```

NAPOMENA: U prvom prikazanom obliku (**Do Until..Loop**), instrukcija prvo testira istinitost izraza. Ako je izraz istinit, izvršava se prva instrukcija posle Loop, a ako je izraz netačan, izvršavaju se instrukcije unutar petlje sve dok izraz ne dobije vrednost istina.

U drugom slučaju (**Do..Loop While**), instrukcija izvršava sve instrukcije unutar petlje, pa tek onda testira uslov. Ako je izraz netačan, ponavljaju se instrukcije u petlji, a ako je tačan, izvršava se prva instrukcija koja dolazi iza petlje.

U trećem prikazanom obliku (**Do While..Loop**) instrukcije prvo se testira *Bulov.izraz*. Ako je izraz tačan, izvršavaju se instrukcije unutar petlje sve dok uslov ne dobije vrednost netačan. Tada se izvršava prva instrukcija izvan petlje.

U četvrtom obliku ove instrukcije (**Do..Loop While**), ona prvo izvršava instrukcije unutar petlje, pa tek onda testira uslov. Ako je izraz tačan, ponavljaju se instrukcije u petlji, dok u suprotnom ide na prvu instrukciju izvan petlje.

U svim slučajevima može se, u okviru petlje iskoristiti instrukcija **Exit Do** za izlaz iz petlje (obično pod nekim uslovom).

PRIMER 1:

```
Do Until testChar = Chr(vbKeySpace)
  lineIndex = lineIndex - 1
  docIndex = docIndex - 1
  testChar = Mid$(printLine, lineIndex, 1)
Loop
```

'smanji lineIndex
'smanji docIndex
'pogledaj prethodni testChar

PRIMER 2:

```
Do
  ima = InStr(s, Chr$(96))
  If ima Then
    Mid$(s, ima, 1) = Chr$(64)
  End If
Loop Until ima = 0
```

PRIMER 3:

```
i = 0
Do While i < 100
  Print i
  i = i + 1
Loop
```

5.8 Načini i principi korišćenja instrukcije Exit

Ponekad je neophodno da "na pola puta" izađete iz petlje ili procedure, tj. pre nego što se petlja ili procedura zatvori na uobičajen način. U ovoj situaciji ćete sigurno pomisliti na nesrećnu instrukciju bezuslovnog grananja GoTo, koja nažalost u *VisualBasic* -u i dalje postoji, a kada je iskoristite odbacili ste sve principe strukturnog programiranja.

Instrukcija Exit može se primeniti u sledeća četiri oblika sintakse:

Exit For

```
For brojač = start To kraj [Step korak]
  [instrukcije]
[Exit For]
[instrukcije]
Next [brojač]
```

Exit Do

Exit Do instrukcija se koristi u sastavu Do..Loop petlje i to na jedan od navedenih načina:

<pre>Do [{While Until} uslov] [instrukcije] [Exit Do] [instrukcije] Loop</pre>	<pre>Do [instrukcije] [Exit Do] [instrukcije] Loop [{While Until} uslov]</pre>
--	--

Exit Sub

Exit Sub instrukcija je nezaobilazna u procedurama Sub, a koristi se u obliku:

```
[Private | Public] [Static] Sub ime [(arg)]
  [instrukcije]
[Exit Sub]
[instrukcije]
```

End Sub

Exit Function

Exit Function je sastavni deo procedure Function u obliku:

```
[Public | Private] [Static] Function ime [(arglist)] [As type]
    [instrukcije]
    [ime = izraz]
[Exit Function]
    [instrukcije]
    [ime = izraz]
End Function
```

U prethodna dva navedena oblika primene instrukcije Exit, programske linije procedure između Exit i End instrukcije se koristi za servisiranje grešaka, što će biti kasnije obrađeno u poglavlju *Servisiranje grešaka*, a kako je proceduralno programiranje neodvojivo od servisiranja grešaka, ovde ćemo prikazati nekoliko načina usklađivanja instrukcije Exit i servisa grešaka:

PRIMER 1:

Primer funkcije **IsExistFile** koja vraća logičku vrednost True, ako traženi fajl **ThisFile** (sa punim imenom staza+ime+ekstenzija) postoji, a u suprotnom False.

```
Function IsExistFile(ThisFile As String) As Boolean
    Filnum = FreeFile
    On Local Error GoTo Err_Exist      'Ako se pojavi greška odvijanje procedure se
    Open ThisFile For Input As Filnum  'nastavlja od linije Err_Exist
    Close Filnum
    IsExistFile = True                 'Dodela vrednosti funkciji kada nema greške
Exit Function                          'Izlaz iz funkcije
Err_Exist:
    IsExistFile = False                'Dodela vrednosti funkciji kada postoji greška
End Function                          'Fizički kraj funkcije
```

PRIMER 2:

Primer funkcije **fExist** kao što je gornja, ali bez korišćenja Exit instrukcije.

```
Function fExist(fName As String) As Boolean
    Dim fNum As Long                  'Deklarisanje promenljive
    fNum = FreeFile                  'Dodela slobodnog broja za otvaranje datoteke
    On Local Error Resume Next        'Ako se greška detektuje procedura se nastavlja
    Open fName For Input As fNum      'Otvaranje datoteke (ispitivanje egistencije)
    Close fNum                        'Zatvaranje iste datoteke
    Select Case Err.Number             'Selekcija slučajeva po broju greške
        Case 0: fExist = True          'kada je broj greške ne postoji - nema greške
        Case Else: fExist = False      'kada je broj greške bilo koji drugi broj-greška
    End Select
End Function                          'Fizički kraj funkcije
```

6 Zajednički rad na izradi aplikacije (9, 10)

Probijanje leda između nas i VisualBasic-a

6.1 Pristup i zahtevi

Aplikacija, koju ćemo zajednički izraditi, ne mora (i ne može) imati projektni zadatak, jer ćemo kroz njenu izradu, pokušati da rešimo probleme i nedoumice koje i sami budete imali pri izradi pojedninačnih zadataka koje ste dobili.

Softver se najčešće projektuje po zahtevima korisnika (investitora), ali je iskustvo i sposobnost programera od ključnog značaja. Korisnik uglavnom teži svom cilju, pa pri tom ne sagledava - šta sve može da izazove problem u samom korišćenju softvera prema iskazanom zahtevu. U toj situaciji programer mora da zadovolji i zahtevima korisnika, a pri tom ne ugrozi uspostavljen standard OS-a.

6.2 Redosled pri kreiranju projekta

Neka vam sledeći redovi posluže kao napomena na naš zajednički razvoj ove aplikacije:

- Otvaranje novog direktorijuma za novi projekat
- Održavanje datoteka projekta na okupu
- Koraci pri izradi aplikacije
 - Postavljanje forme
 - Dodavanje kontrola na formu
 - Podešavanje svojstava kontrola
 - Izvršavanje projekta
 - Dodavanje kôda u procedure događaja
- Snimanje datoteka projekta
- Prevođenje projekta
- Prenos projekta na drugi računar
- Servisiranje grešaka
 - Greške nivoa procedure
 - Greške nivoa modula
- Kreiranje instalacione verzije programa

Možda neke od navedenih postupaka nećete moći obaviti sada, ali ne zaboravite da ih obavite po izlaganju teme.

7 Upotreba objekata za poruke i jednostavan dijalog (I I-I2)

Kako se pred korisnika postavlja najprostiji dijalog

7.1 Funkcija (i instrukcija) MsgBox

MsgBox funkcija prikazuje poruku u malom prozoru sa očekivanjem jedne akcije korisnika (klik na taster) i vraća vrednost tipa Integer kojom se upravo otkriva navedena akcija.

SINTAKSA:

MsgBox(prompt[, buttons] [, title] [, helpfile, context])

gde je:

- prompt** Obavezan string izraz koji će biti prikazan kao poruka u prozoru, sa maksimalnom dužinom oko 1024 znaka, zavisno od širine korišćenih karaktera. Kada poruku treba prelomiti u više redova koriste se specijalni znaci: Chr(13), Chr(10), ili u kombinaciji Chr(13) & Chr(10) između delova poruke.
- buttons** Opcioni numerički izraz kojim se deklariše prikazana ikonica i ekranski tasteri uz poruku. Ako je ovaj izraz izostavljen koristi se 0.
- title** Opcioni string izraz koji će biti prikazan u naslovu prozoru, a ako je izostavljen, na ovo mesto se postavlja ime aplikacije projekta.
- helpfile** Opcioni string izraz koji identifikuje Help-fajl koji će se uz prikazanu poruku vezati. Ako je ovaj argument naveden onda je sledeći parametar obavezan, da ostvari vezu korisnika sa navedenim Help-om.
- context** Opcioni. numerički izraz o delu Help-fajla u kojem se nalazi objašnjenje za traženu pomoć.

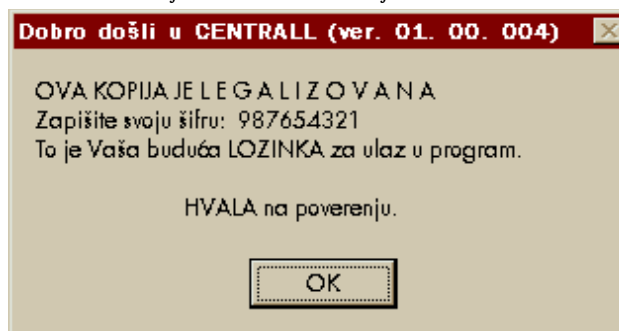
Argumenti za postavljanje vidljivih tastera u MsgBox objektu:

Konstanta	Vrednost	Prikazuje
vbOKOnly	0	sa OK taster
vbOKCancel	1	OK i Cancel tasteri
vbAbortRetryIgnore	2	Abort, Retry i Ignore tasteri
vbYesNoCancel	3	Yes, No i Cancel tasteri
vbYesNo	4	Yes i No tasteri
vbRetryCancel	5	Retry i Cancel tasteri
vbMsgBoxHelpButton	16384	Prikaz Help tastera
vbCritical	16	Critical ikona
vbQuestion	32	Ikona pitanja
vbExclamation	48	Ikona upozorenja
vbInformation	64	Ikona informisanja
vbDefaultButton1	0	Prvi taster je default.
vbDefaultButton2	256	Drugi taster je default.
vbDefaultButton3	512	Treći taster je default.
vbDefaultButton4	768	Četvrti taster je default.
vbApplicationModal	0	Izvršavanje aplikacije se obustavlja dok se ne odgovori na poruku
vbSystemModal	4096	Rad O.S.-a se obustavlja dok se ne odgovori na poruku
VbMsgBoxSetForegroun	65536	Postavlja prozor poruke u pozadinu tekućeg prozora
vbMsgBoxRight	524288	Poravnanje teksta udesno
vbMsgBoxRtlReading	1048576	Specificira da bi tekst trebalo da se čita s desna u levo (kao hebrejski i arapski)

Prvom grupom vrednosti (0-5, 16384) opisuje se broj i tip tastera koji će biti prikazani; druga grupa (16, 32, 48, 64) opisuje vrstu ikone koja će biti prikazana uz poruku; treća grupa (0, 256, 512, 768) definiše koji od tastera je default taster (aktivira se na <Enter> i <Space>); četvrta grupa (0, 4096) definiše mod u kojem se poruka prikazuje; peta grupa (65536, 524288, 1048576) definiše efekte teksta. Pored naziva konstante, na isti način mogu se koristiti i same njihove vrednosti, koristeći sabiranje. U ovom slučaju treba voditi računa da se **iz jedne grupe može koristiti samo jedna konstanta**.

Za detekciju akcije (izbora) korisnika, mogu se koristiti sledeće vrednosti koje vraća ova funkcija:

konstanta	vrednost	akcija - pritisnut taster
vbOK	1	OK
vbCancel	2	Cancel
vbAbort	3	Abort
vbRetry	4	Retry
vbIgnore	5	Ignore
vbYes	6	Yes
vbNo	7	No



PRIMER 1: za instrukciju *MsgBox*

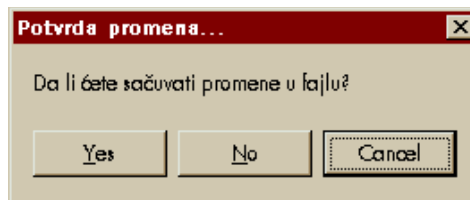
```
MsgBox "OVA KOPIJA JE L E G A L I Z O V A N A" & vbCrLf & _
  "Zapišite svoju lozinu: " & Str(Password1) & vbCrLf & _
  "To je Vaša buduća identifikacija za ulaz u program." & vbCrLf & vbCrLf & _
  "HVALA na poverenju.", , _
  "Dobro došli u CENTRALL (ver. " & _
  Format(App.Major, "00. ") + Format(App.Minor, "00. ") + _
  Format(App.Revision, "000") & ")"
```

Izvršavanjem ove instrukcije dobija se poruka kao na prethodnoj slici.

PRIMER 2 za funkciju *MsgBox*

```
Select Case MsgBox("Da li ćete sačuvati promene u fajlu?", _
  vbYesNoCancel + vbDefaultButton3, "Potvrda promena...")
Case vbYes
  mnuFileSave_Click
Case vbCancel
  Exit Sub
End Select
```

Izvršavanjem funkcije (u osenčenom delu primera) dobija se poruka kao na slici desno.



7.2 Funkcija InputBox

InputBox funkcija prikazuje tekst poruku u malom prozoru, čeka da korisnik unese tekst i klikne na taster "OK", a funkcija dobija string vrednost - sadržaj Textbox-a.

SINTAKSA:

InputBox(prompt[, title][, default][, xpos][, ypos][, helpfile, context])

Opis argumenata:

- prompt** Obavezan string izraz koji će biti prikazan kao poruka u prozoru, sa maksimalnom dužinom oko 1024 znaka, zavisno od širine korišćenih karaktera. Kada poruku treba prelomiti u više redova koriste se specijalni znaci: Chr(13), Chr(10), ili u kombinaciji Chr(13) & Chr(10) između delova poruke.
- title** Opcioni string izraz koji će biti prikazan u naslovu prozoru, a ako je izostavljen na ovo mesto biće postavljeno ime aplikacije - projekta.
- default** Opcioni string izraz kojim se postavlja kao default² sadržaj u Textbox-u. Ako propustite ovaj argument Textbox će se pojaviti kao prazan.
- xpos** Opcioni numerički izraz kojim se, u twip³-ovima, definiše horizontalnu udaljenost leve ivice prozora ovog dijaloga od leve ivice screen⁴-a, a ako je izostavljen, dijalog će biti centriran po horizontali.
- ypos** Opcioni numerički izraz kojim se, u twip-ovima, definiše horizontalnu udaljenost gornje ivice prozora ovog dijaloga od gornje ivice screen-a, a ako je izostavljen, dijalog će biti centriran po vertikali.
- helpfile** Opcioni string izraz koji identifikuje Help fajl (*.HLP) koji će se uz prikazanu poruku vezati. Ako je ovaj argument naveden onda je sledeći parametar obavezan, da ostvari vezu korisnika sa navedenim Help-om.
- context** Opcioni numerički izraz o delu Help fajla u kojem se nalazi objašnjenje za traženu pomoć.

² default - podrazumevano

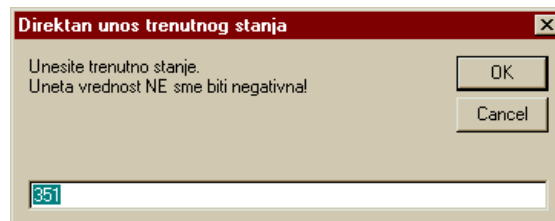
³ 1 inč=1440 Twip-ova, a 1cm=567.

⁴ raspoloživi radni prostor na ekranu monitora

NAPOMENA: Kada je pristup Help fajlu obezbeđen tada, pritiskom na taster F1, korisnik dobija pregled teme u Help fajlu, pridružene argumentom context. Navigacija kroz Help je obično krajnje jednostavno, a izlaz iz ovog servisa uvek je obezbeđen pritiskom na taster <ESC>.

PRIMER:

```
Do
    s(1) = LTrim(InputBox("Unesite trenutno stanje." & vbCrLf &
        "Uneta vrednost NE sme biti negativna!", _
        "Direktan unos trenutnog stanja", txtMain(3)))
Loop Until Val(s(1)) >= 0
```



7.3 Visual Basic for Applications (VBA)

Kako najlakše proniknuti u tajne paketa MS Office 97

Još davne 1987, iz Microsoft-a je najavljen zajednički script jezik za sve njihove aplikacije, ali obećanje je u potpunosti ispunjeno tek pošto je objavljen *VisualBasic for Applications 5* (VBA5) uz paket *MS Office 97*. Kao razvojna platforma, odnosno skrip jezik, za izradu aplikacija za programe ovog paketa (*Access*, *Excel*, *Word*, *PowerPoint*,...) je uključen neophodni deo *VisualBasic*-a 5.0, pa je po njemu dobio i ime.

Priča počinje još od prve verzije *MS Access*-a (za projektovanje i korišćenje baza podataka), gde bi projektovanje bez podrške nekog razvojnog alata za rad u kôdu bio prilično ograničen. Sredinom 1990 godine u Microsoftu započinje rad na na ovoj, delimično objektno orjentisanoj, verziji *Basic*-a, koji je u prvo vreme nosio ime *Object Basic*, a zatim preimenovan u *VisualBasic for Applications*. Prva verzija ovog razvojnog alata bila je 1993 godine uključena u *Excel 5.0* i pokazala se kao izuzetno jednostavna i prihvaćena od strane "običnih" korisnika, za koje je i namenjen.

VBA2 sa *Excel*-om i *Access*-om 95 je otišao korak dalje po pitanju performansi, da bi postao deo *VisualBasic*-a 4.0. Sa pojavom *Office*-a 97 i VBA5, Microsoft je priveo kraju svoj dugoročni plan script jezika uključenog u svoje programe. Tako sada, koristeći VBA, možete upravljati *Excel*-om, *Access*-om, *PowerPoint*-om, *Outlook*-om i, naravno, *Word*-om. Šta više Microsoft deli licencu za jezik, pa VBA možemo pronaći i u programima drugih proizvođača - *AutoCad R14* i *Visio Professional 4.5* su samo prvi primeri. Sve programe koji poseduju VBA makro jezik prepoznaćete po oznaci *VisualBasic Technology*.

Microsoft i VBA ovde ne staju. Sledeći očekivani korak u razvoju VBA je njegovo uključivanje kao sastavnog dela operativnog sistema, o čemu se u *Microsoft*-u još razmatraju troškovi i eventualni efekti.

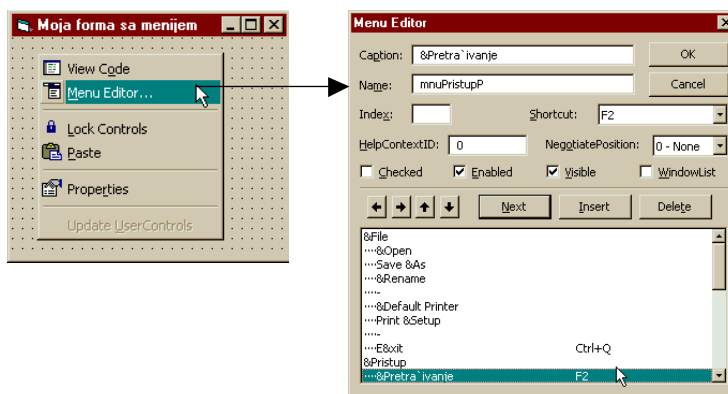
Pravljenje makroa i programiranje u VBA je - koliko korisno, toliko i jednostavno, a svo znanje stečeno u *VisualBasic*-u možete iskoristiti i u VBA i upoznati programe *Office*-a iznutra.

NAPOMENA: Zajedno ćemo otvoriti neke od programa *MS Office*-a, kreirati par procedura i videti kako to sve dobro radi i od kolike koristi može biti svakom korisniku ovog paketa programa. Videćete nekoliko aplikacija u *Word*-u, *Excel*-u, *PowerPoint*-u i *Access*-u, koje mogu da znatno ubrzaju rad na mnogim poslovima poslovanja.

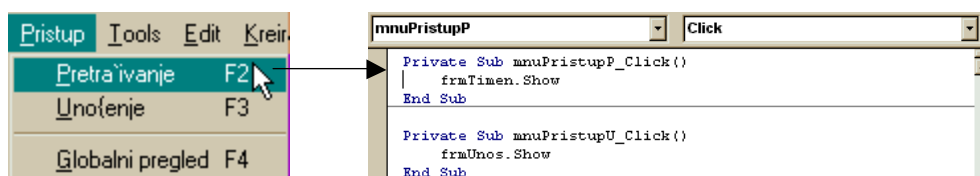
8 Kreiranje sistema menija (13-14)

Jedan od najefikasnijih servisa VisualBasic-a

8.1 Prozor za kreiranje menija



8.2 Prikazivanje kôda stavke menija



8.3 Kontrolisani izlaz iz aplikacije

```
Private Sub Form_Unload(Cancel As Integer)
    Cancel = 1
    mnuFileX_Click
End Sub

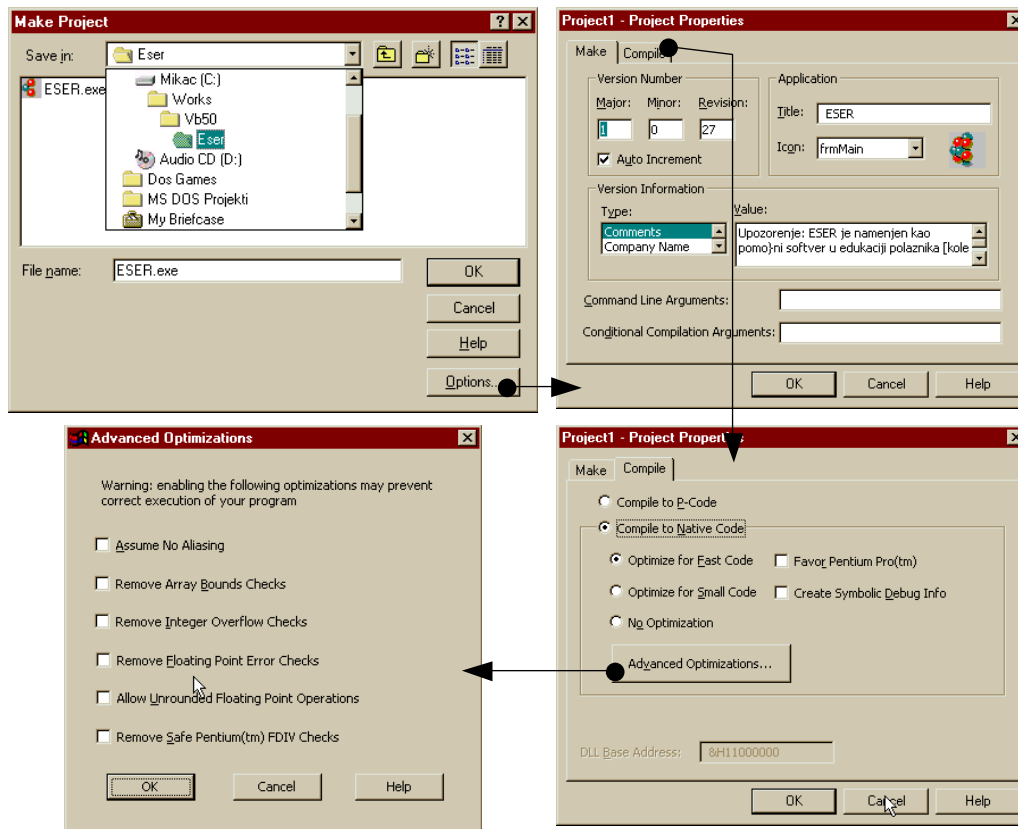
Public Sub mnuFileX_Click()
    KrajRada
End Sub

Private Sub KrajRada()
    Msg = "Kraj rada u programu?"
    Style = vbYesNo + vbInformation + vbDefaultButton1
    Title = "Exit..."
    Help = "Timen.hlp"
    Ctxt = 1000
    If MsgBox(Msg, Style, Title, Help, Ctxt) = 6 Then
        NewProgramIni
        Open MyDesk For Output As #1
        For i = 0 To 5
            Write #1, SetGro(i)
        Next
        Close
    End If
End Sub
```


9 VisualBasic Setup Wizard (13-14)

Kako kreirati instalacionu verziju aplikacije

9.1 Kreiranje izvršnog programa projekta



9.2 Provera integriteta i lokacije komponenata

Potrebne datoteke i/ili baza podataka

Ono što može da vas iznenadi je da zaboravite koje ste sve datoteke (inicijalne, tekstualne, sekvencijalne, i sl.) koristili u vašem projektu. Sve ove datoteke korisnik aplikacije mora imati da bi jednako dobro radio u vašem softveru. Ako je kreirana aplikacija prosta, ne poziva nikakve datoteke niti ima *Link* na njih, ovo vam neće trebati, a ako nije tako, dobro je da napravite popis svih pozivanih datoteka.

Dok razvijate vaš projekat, sve datoteke pozivajte koristeći funkciju `App.Path` ili sa floppy disk-a (demo verzije). Na koristite apsolutni path odnosno datoteku sa stazom npr. "C:\000\JA\MOJFAJL.dat", jer ne možete očekivati da korisnik aplikacije raspoređuje vaše datoteke po disku i pravi "situaciju kao kod vas". Ako koristite funkciju `App.Path`, sve datoteke ćete moći postaviti u direktorijum aplikacije (ma kako se zvao i gde bio) i sve dublje direktorijume.

Sve ove datoteke (i eventualno baza podataka) treba da budu u sastavu instalacione verzije vašeg programa, sa ko-
rektno zadanom stazom odredišta, da se korisnik ne bi iznenadio.

Fontovi

Sve fontove koje ste koristili u vašem projektu (ne koristite više od dva-tri različita fonta) iskopirajte u jedan novi direktorijum (npr. \Fonts\) na vašem direktorijumu projekta. U situaciji kada ne znate koje ste sve fontove u projektu koristili, iskoristite neki običan editor teksta, otvorite module svih vaših formi (*.frm), pa pretražite sve njih tražeći tekst 'Name = ' sa 12 praznih mesta. U produžetku ovog teksta stoji ime fonta, kao npr.:

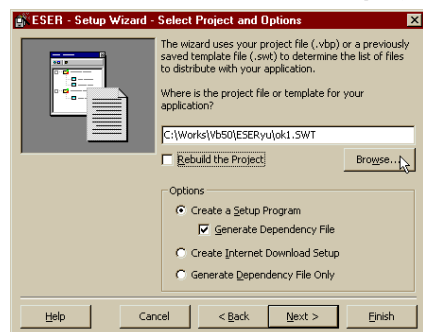
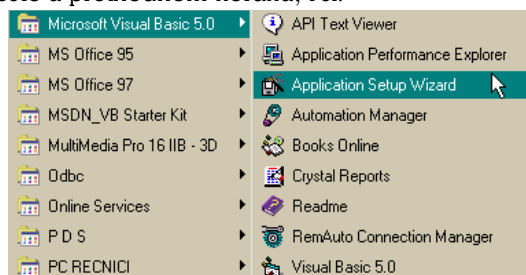
Name = "Arial"

9.3 Pokretanje SetupWizard -a

VisualBasic Setup Wizard se koristi za brzo i lako kreiranje SETUP programa aplikacije koju ste kreirali u *VisualBasic*-u. Kao i ostali čarobnjaci, *Setup Wizard*, vam daje sve potrebne informacije o onom šta možete u njemu da uradite, možete se komforno kretati kroz sve njegove segmente, promeniti nešto u prethodnom koraku, i sl.

U velikom broju slučajeva (prosta aplikacija) *Setup Wizard* će odrediti i odabrati sve umesto vas, vaše je samo da odabrane opcije potvrdite.

Da biste pokrenuli *Setup Wizard*, pronađite i pokrenite stavku *Application Setup Wizard* u grupi *Microsoft Visual Basic 5.0*, vašeg start menija, posle čega ćete dobiti prozor kao na drugoj slici. Ovde ćete moći otvoriti neki od postojećih fajlova *Setup Wizard*-a (*.SWT) ili neki od *VisualBasic* projekata (*.VBP).

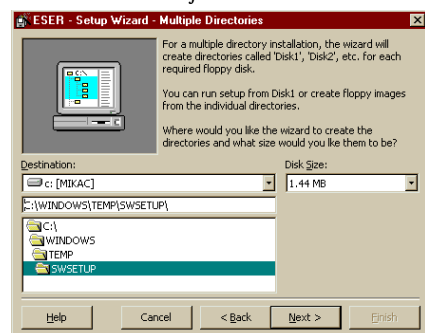


Zatim ćete u novootvorenom prozoru *Distribution Method* moći da odaberete da li ćete vašu instalacionu verziju distribuirati preko:

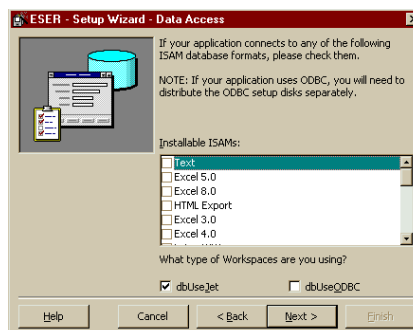
- floppy disketa,
- jednog direktorijuma ili
- više direktorijuma koji će, po veličini, odgovarati vašim disketama.



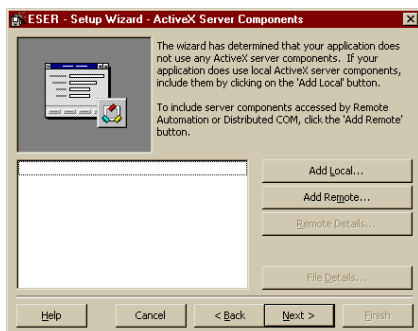
Potom ćete odabrati kapacitet floppy disketa i/ili destinacioni direktorijum za vašu instalaciju.



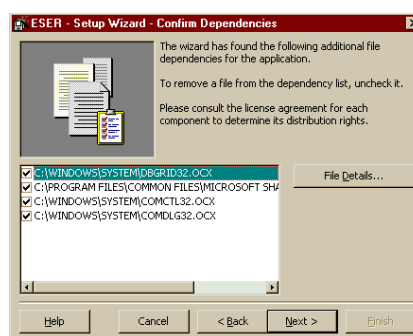
Izabrati ISAM Database format
(ako ste koristili)



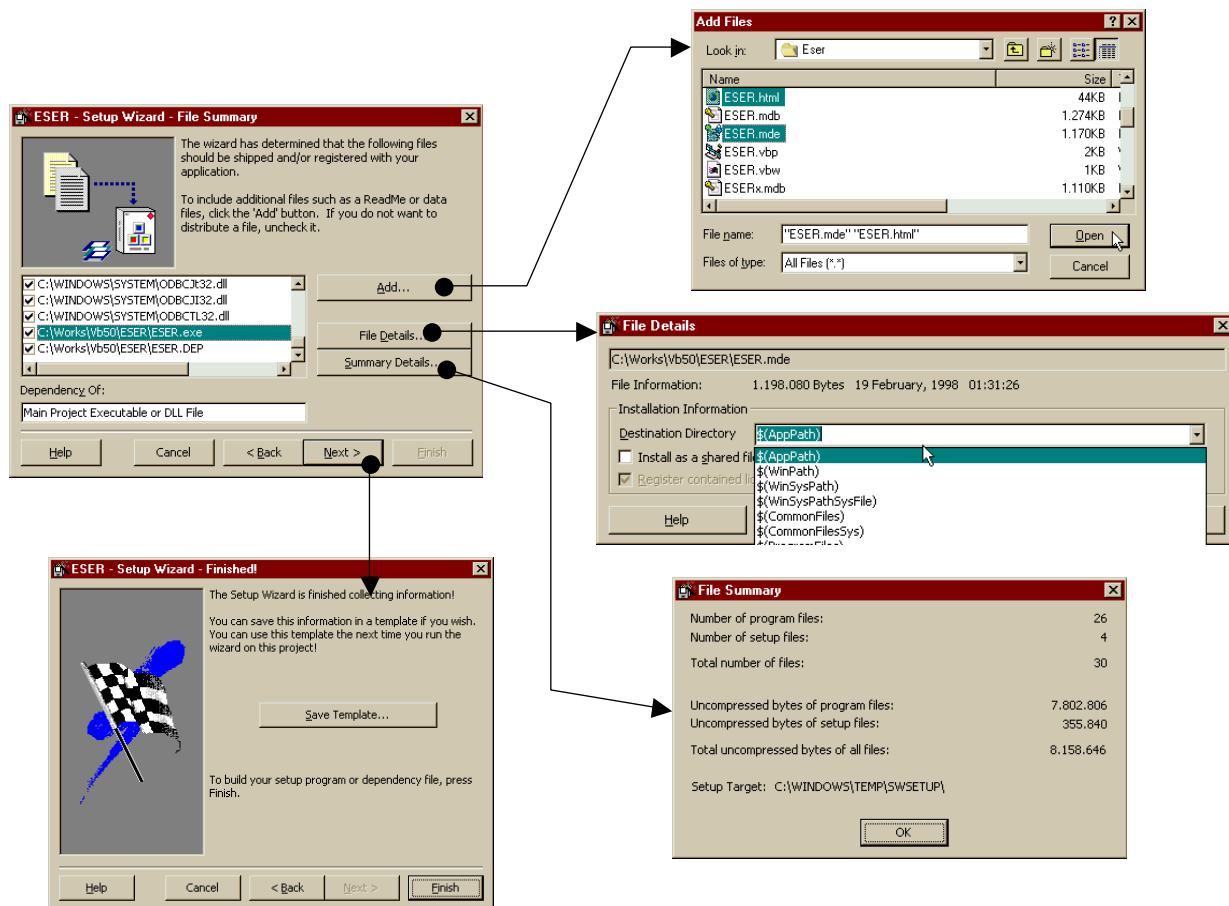
Dodati ActiveX Server Komponenti:



Pridružiti sve fajlove
(grafičkih kontrola)
od kojih aplikacija zavisi



I konačno, dodati korišćene sekvencijalne datoteke, fontove i ostale fajlove koje *Setup Wizard* nije prepoznao kao po-

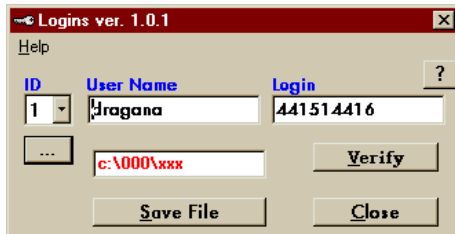


trebne za instalaciju, pa odrediti njihovo odredište.

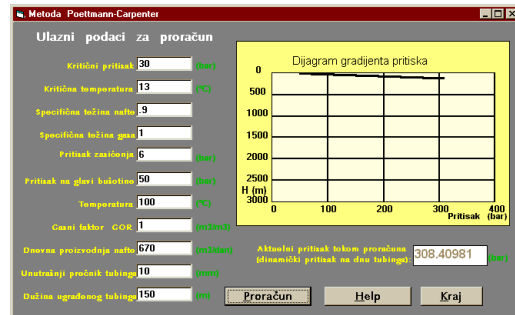
Sve opcije i veze na dodane datoteke mogu se na kraju sačuvati u *.swt fajl., a instalacione diskete distribuirajte bu-
dućim korisnicima vaše aplikacije.

Dodatak A

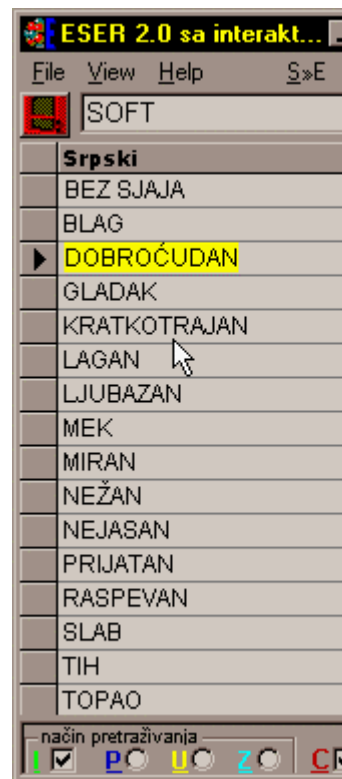
Neke aplikacije razvijene u VisualBasic-u



Aplikacija **LOGINS** razvijena je za situaciju kada izvesnoj aplikaciji treba ograničiti pristup na nekoliko korisnika. Svakom korisniku se, pored imena, dodeljuje i lozinka pristupa, svi uneti podaci se konvertuju da se ne bi mogli lako otkriti. Pravo korišćenja ove aplikacije treba da ima



Aplikacija **POETMAN** je namenjena za korišćenje ispitivanja ponašanja nafte na eksploatacionim bušotinama. Omogućava kontrolu unetih podataka i daje grafički prikaz gradijenta pritiska.



ESER je aplikacija za prevod reči sa engleskog na srpski jezik i obrnuto. Razvijena je kao program za edukaciju polaznika naše Škole i pomoć u slučaju problema sa Engleskim jezikom prilikom vežbi.

Posle svake unete promene tražene reči, ESER će pokušati da vam pronade njene prevode.

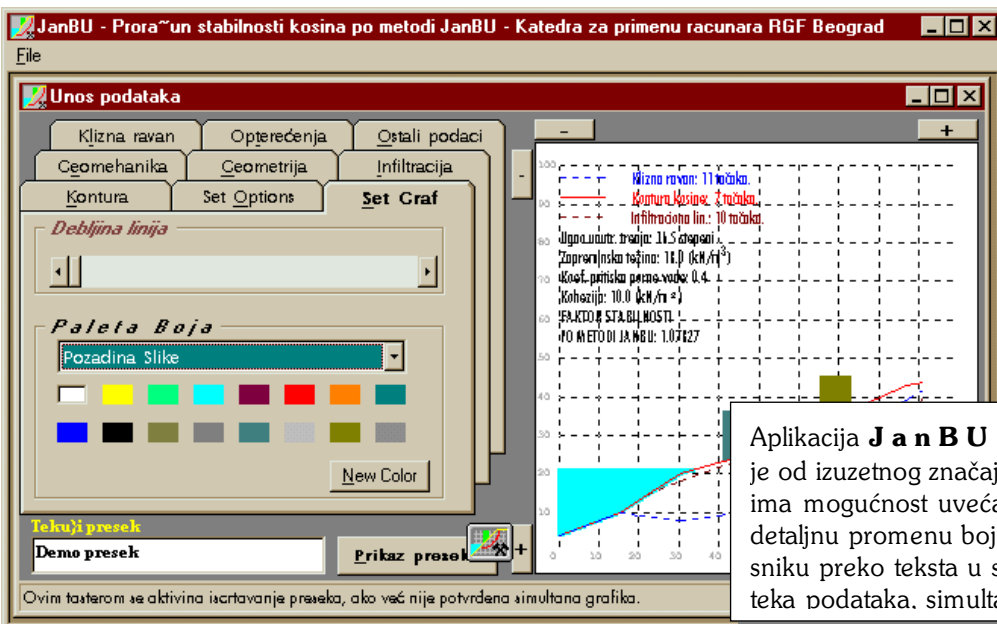
ESER ima mogućnost Englesko-srpskog (Ctrl+E, odnosno Srpsko-engleskog (Ctrl+S) prevoda na dva osnovna načina:

- 1) prikaz sinonima koje ste tražili kao cele reči
- 2) prikaz onih sinonima koji zadovoljavaju uneti početak reči.

ESER može sinonime da prikazuje u obliku tabele, ali se preporučuje prikaz u minimiziranom obliku na Taskbar-u uz korišćenje Clipboard-a.

To znači da ćete ovu aplikaciju minimizirati, dok radite u bilo kojoj Windows aplikaciji, pa kada vam zatreba prevod neke reči dovoljno će biti da je iskopirate (Ctrl+C), a ESER će njene sinonime prikazati na svom OnLine tasteru na Taskbar-u.

Instalaciona verzija programa staje na 5 disketi i može se dobiti preko autora.



Aplikacija **JanBU** razvijena je prema istoimenoj metodi za proračun faktora stabilnosti kosina, a što je od izuzetnog značaja za odvijanje eksploatacije na površinskim kopovima. Pored kvalitetnog interfejsa ima mogućnost uvećanja/smanjenja grafičkog prikaza, print preview, štampanje u dva nivoa kvaliteta, detaljnu promenu boje delova grafika, definisanje nove boje iz Windows palete, diskusiju i pomoć korisniku preko teksta u statusnoj liniji, otvaranje postojećih, čuvanje i kreiranje novih sekvencijalnih datoteka podataka, simultan grafički prikaz, čuvanje svih odabranih opcija pri radu (boje, položaji i veličina

Contacts ver. 1.01... Copyright Mikac '97

Osoba: **Pera Vučić** Tip kontakta: **FAX**
 Datum kontakta: **06.02.98** Telefon
 Pismo

Napomena:
 Tražio je našu saradnju i ideje za novi časopis koji će se baviti temama iz informatike, ali na jednom nivou bliskom većini korisnika kod nas i problemima koje oni svakodnevno sreću.

Datum	Tip Kontakta	Napomena
08.06.96	Sastanak	Razgovorali smo o njegovim investicijama
18.07.96	Telefon	Ponovo je pozvao da se uveri da je sve
01.01.97	FAX	Zainteresovana je za posao u preduzeću
06.02.98	Pismo	Tražio je našu saradnju i ideje za novi
10.03.97	Telefon	Zakazana reklama kreće od petka
10.03.97 2:24:4	Pismo	Dolazi na sastanak svih producenata

Kretanje kroz bazu

Novi Prethodni Sledeći Obriši Exit

Pretraživanje baze PROGRESIVNO...

Pretraži: **Pavlović** Copy to Second Move to Second

Ime: **Svetislav** Zanim: **dipl. inž. rud.** Telefon u stanu: **015/32-241**

Firma: **DD "Zorka" nemetali i građevinski mat.** Funkcija: **Rukovodilac sektora**

Telefon u firmi: **015/31-754** Fax: **015/24-072**

Adresa: **Narodnih heroja bb** Mesto: **15000 Šabac**

Copy Cut Paste

Brisi slog Štampanje Exit

Prethodni 792 Sledeći

Timen30

(iz koga je dat samo jedan od prozora) je prvi Windows program autora ovog teksta, razvijen u *VisualBasic-u*, i ponos u pogledu postignutih kvaliteta:

- rad na random datotekama u mrežnom okruženju,
- višekorisnički i ravnopravan rad (i iz istoimenog DOS programa) na istoj datoteci potuno ravnopravno,
- istovremeno korišćenje više datoteka (primarna-sekundarna) u slučaju potreba kopiranja ili prebacivanja podataka,
- rešen problem kod pretraživanja (velika-mala) YU slova,...

Prvobitno ovaj program je trebalo da bude samo **Telefon-ski IMENik**, ali je, simultanim korišćenjem i razvijanjem, dostigao visok kvalitet te je korišćen za vođenje evidencija, pretraživanje, slanje pisama, pregled i štampanje lista učesnika više seminara i zaposlenih u firmama.

MiniTim

predstavlja najmanju moguću aplikaciju za vođenje adresara i telefonskog imenika sa mogućnostima simultanog pretraživanja. Aplikacija koristi kriptovanu Access bazu podataka, što znači da baza nije pod šifrom, ali se realni podaci iz nje mogu čitati, kao i upisivati, samo ovom aplikacijom.

Za administriranje korisnika ove aplikacije koristi se navedena aplikacija Logins, a bez imena i lozinke ulaz u aplikaciju nije moguć, što nalazi svoju primenu kod adresara kome treba da pristupa ograničen broj korisnika.

MiniTim ver. 1.0.1.

File Edit Tools Format

Osoba: **Mikanović Radenko** Save Novi Obriši Find

Prazime, ime: **Mikanović** Radenko Firma: **Print93**
 Adresa: **Majke Jevrosime 38** Titula: **dipl.ing.rud**
 Odeljenje: **Razvojni studio**
 Grad, država: **Beograd** YU Telefon: **011/3223-154**
 FAX: **011/3223-154**
 E-mail: **office@bbos.co.yu**

Find
 Vrata podataka
 Prazime
 Cancel
 Sledeći

Kategorije
☐ Sa YU slovima ☒ Dva podataka

Kretanje

Program VODA... (modeliranje voda)

Exit Window

1 Unos opštih podataka
 2 Podaci o bušotinama
 3 Unos podataka o konturama

KRC

Naziv modelirajućeg obeležja
KOTE KROVINE

Odgovorno lice
Dipl. Ing. Ilija Petrović

Xo: **0.000** (m)
 Yo: **0.000** (m)
 dx: **20.000** (m) dy: **20.000** (m)

Broj bušotina: **9** Radijus uticaja: **150.000**

Matrica k

Broj bušotine u sistemu: **1**

Oznaka bušotine: **B-01** Modelirajuće obeležje: **170**

X = **500.000**
 Y = **500.000**
 Z = **183.000**

Opšti podaci Kontura

Unos podataka o konturama

Istočna kontura

Oznaka bušotine: **B-01** Broj bušotine u sistemu: **1**

X = **500.000**
 Y = **500.000**

Next

Contacts

je relativno mala aplikacija za vođenje podataka o kontaktima sa klijentima i saradnicima. Urađena je po ugledu na slične aplikacije pod DOS-om i oslanja se na Access bazu podataka.

Ova aplikacija se pokazala kao korisna, kako za poslovne sekretare, tako i za sve one koji vode računa da svoja obećanja ispune ne opterećujući se suvišnim podacima o zakazivanjima.

Program **VODA** namenjen je za matematičko modeliranje i grafičko predstavljanje stanja podzemnih voda. U ovoj aplikaciji autor je značaj posvetio formatizovanju numeričkih vrednosti koje se unose kao tekst, prikaz aplikacije u MDI obliku, otvaranje sekvencijalnih datoteka podataka,... Matematički model, primenjen u ovoj aplikaciji, odudara od relativno visokog kvaliteta interfejsa. Autora je imao zadatak da uradi aplikaciju za Windows, na matematički model starije aplikacije urađene za DOS, koji nije trebalo menjati. Neke izmene koje je autor, ipak, morao da uvede, vezane su za deklarisanje promenljivih i rad sa datotekama podataka, što u staroj verziji programa nije bilo omogućeno.

Dodatak B

Grafički prikaz sistema padajućih menija

File	
New Project	Ctrl+N
Open Project...	Ctrl+O
Add Project...	
Remove Project	
Save Project	
Save Project As...	
Save frmEngleskaRec.frm	Ctrl+S
Save frmEngleskaRec.frm As...	
Print...	Ctrl+P
Print Setup...	
Make ESER.exe...	
Make Project Group...	
1 ..\..\..\Works\vb50\ESER\ESER.vbp	
2 ..\..\..\VB4PLUS\SRC\WEATHER\Ciswww.vbp	
3 samples\Visdata\Visdata.vbp	
4 samples\PGuide\MsFlexGd\Flex.vbp	
Exit	Alt+Q

- Kreiranje novog projekta
- Otvaranje postojećeg projekta
- Dodavanje projekta u grupu
- Uklanjanje projekta iz grupe
- Čuvanje projekta (i svih modula)
- Promene opcija čuvanja projekta
- Čuvanje tekućeg modula
- Promene opcija čuvanja modula
- Slanje na štampač
- Postavljanje opcija štampanja
- Kreiranje *.EXE fajla
- Kreiranje grupe projekata
- Poslednji otvarani projekti na raspolaganju
- Kraj rada u VisualBasic -u

Edit	
Undo	Ctrl+Z
Redo	Ctrl+Y
Cut	Ctrl+X
Copy	Ctrl+C
Paste	Ctrl+V
Paste Link	
Delete	Del
Select All	Ctrl+A
Find...	Ctrl+F
Find Next	F3
Replace...	Ctrl+H
Indent	Tab
Outdent	Shift+Tab
Insert File...	
List Properties/Methods	Ctrl+J
List Constants	Ctrl+Shift+J
Quick Info	Ctrl+I
Parameter Info	Ctrl+Shift+I
Complete Word	Ctrl+Space
Bookmarks	

- Poništavanje poslednje akcije
- Poništavanje poslednjeg "Undo"
- Isecanje sa memorisanjem
- Kopiranje (memorisanje)
- Lepljenje (memorisanog)
- Lepljenje sa "Link"om
- Brisanje
- Selektovanje svega (u prozoru)
- Zadavanje teksta za pretraživanje
- Ponavljanje pretraživanja
- Zamena jednog teksta drugim
- Uvlačenje linija (za Tab)
- Izvlačenje linija (za Tab)
- Ubacivanje sadržaja tekstualnog (*.txt, *.bas, *.cls) fajla
- Lista Osobina/Metoda
- Lista konstanti
- Informacije o objektu
- Informacije o argumentu
- Auto dovršavanje započete "reči"
- Rad sa markerima u tekstu

Run	
Continue	F5
Start With Full Compile	Ctrl+F5
Break	Ctrl+Break
End	
Restart	Shift+F5

- Pokretanje/Nastavak izvršavanja
- Start sa punim kompajliranjem
- Obustavljanje rada programa (pauza)
- Prekid rada programa (kraj)
- Ponovni start (kada je obustavljen rad)

Tools	
Add Procedure...	
Procedure Attributes...	
Menu Editor...	Ctrl+E
Options...	

- Dodavanje procedure
- Atributi procedure
- Editor padajućeg menija
- Podešavanje opcija VisualBasic -

Format	
Align	
Make Same Size	
Size to Grid	
Horizontal Spacing	
Vertical Spacing	
Center in Form	
Order	
Lock Controls	

- Poravnavanje selektovanih kontrola
- Dodela zajedničkih dimenzija
- Dimenzije kontrola uskladiti sa mrežom
- Horizontalna udaljenost kontrola
- Vertikalna udaljenost kontrola
- Centriranje selektovane grupe kontrola
- Raspored kontrola po dubini
- Zaključavanje grafičke kontrole

Project	Format	Debug	Run	Tools
Add Form				
Add MDI Form				
Add Module				
Add Class Module				
Add User Control				
Add Property Page				
Add User Document				
Add ActiveX Designer				
Add File...			Ctrl+D	
Remove frmEngleskaRec.frm				
References...				
Components...			Ctrl+T	
ESER_Project Properties...				

Debug	Run	Tools	Add-Ins	Window	Help
Step Into					F8
Step Over					Shift+F8
Step Out					Ctrl+Shift+F8
Run To Cursor					Ctrl+F8
Add Watch...					
Edit Watch...					Ctrl+W
Quick Watch...					Shift+F9
Toggle Breakpoint					F9
Clear All Breakpoints					Ctrl+Shift+F9
Set Next Statement					Ctrl+F9
Show Next Statement					

Help
Microsoft Visual Basic Help Topics
Books Online...
Obtaining Technical Support...
Microsoft on the Web
About Microsoft Visual Basic

View	Project	Format	Debug	Run
Code				
Object			Shift+F7	
Definition			Shift+F2	
Last Position			Ctrl+Shift+F2	
Object Browser			F2	
Immediate Window			Ctrl+G	
Locals Window				
Watch Window				
Call Stack...			Ctrl+L	
Project Explorer			Ctrl+R	
Properties Window			F4	
Form Layout Window				
Property Pages			Shift+F4	
Toolbox				
Color Palette				
Toolbars				

Window	Help
Split	
Tile Horizontally	
Tile Vertically	
Cascade	
Arrange Icons	
1 ESER_Project - frmEngleskaRec (Code)	
2 ESER_Project - frmEngleskaRec (Form)	

Add-Ins	Window	Help
Visual Data Manager...		
Report Designer...		
Add-In Manager...		